



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

CONTROL AND SIGNAL PROCESSING LABAROTARY

Lab Manual

ICE 3241
VI SEM, B.TECH. in EnIE

NAME: _____

REG NO: _____

SECTION: _____

SCHOOL OF ELECTRICAL ENGINEERING

January – 2026



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
A Constituent Unit of MAHE, Manipal.

CERTIFICATE

This is to certify that the Laboratory Manual for lab titled CONTROL AND SIGNAL PROCESSING LABAROTARY (ICE 3241) from Mr./Ms. _____ with Reg. No: _____ of sixth semester of B.Tech. Electronics & Instrumentation Engineering for the academic year 2025-2026 has been submitted as per laboratory course requirements, which has been evaluated and duly certified.

Place:

Date:

Lab In-Charge

SCHOOL OF ELECTRICAL ENGINEERING

January 2026



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

A Constituent Unit of MAHE, Manipal.

CONTROL AND SIGNAL PROCESSING LABAROTARY (ICE 3241)

Exp No.	Title	Faculty Signature
<u>Cycle -1: Software Experiments</u>		
L0	Familiarization with MATLAB	
L1	a) Time domain Specifications and steady state errors b) Frequency response and frequency domain Specifications c) Stability analysis-Root locus, Bode Plot	
L2	State space analysis, controllability, observability and Pole placement	
L3	Proportional and lag, lead, lag-lead compensator design	
L4	a) Signal Generation and Sampling Rate Conversion b) Convolution of Signals c) Z transform	
L5	a) DFT b) FIR Filter Design c) IIR Filter Design	
<u>Cycle -2: Hardware Experiments</u>		
L6	a) Speed control of DC motor using FOPDT model b) Position control of DC motor using FOLIPD model	
L7	Oven parameter identification and temperature control	
L8	a) Lag compensator design b) Lead compensator design	
L9	DSP Hardware Experiments 1: Signal Processing	
L10	DSP Hardware Experiments 2: FIR and IIR filters	
L11-12	Mini Project	

Evaluation Plan:

Continuous Evaluation: **60%** (Preparation, Lab performance, Writeup, Assignments and Regularity, Mini Project)

End Semester Lab Exam: **40%**

L0 - Familiarization with MATLAB Control system toolbox

AIM: To enter, display and transform to different forms of transfer function in continuous time

Common commands and tips for running MATLAB codes:

MATLAB commands:

clc % clears the command window

clear all % clears the workspace

close all % closes all the opened figures

To check value of any variable, enter variable in command window and hit enter. Alternately you can also double click the variable name from the workplace.

While running the code, a semicolon (;) character at the end of line, does not print the variable value in the command window after executing. All the lines without a semicolon (;) character at the end of the line get printed in the command window with their current value.

Do not use command names or reserved keywords as variable or file names.

Problem-1

(a) Consider a plant transfer function $G(s) = \frac{s^3 + 5s^2 + 4s + 6}{4s^3 + 7s^2 + 12s + 9}$. Define the transfer function using two ways.

Open the editor window and a new file. Type the following MATLAB code.

%(a) To enter a transfer function

```
num=[1 5 4 6];
```

```
den=[4 7 12 9];
```

```
G1=tf(num,den)
```

Then save the file as *.m file (* is any file alphanumeric name, must be other than inbuilt function/library function name). Now open command window. Type the file name (without extension) at the command prompt and enter. File will compile and run if there is no error. If there is some error debug the code in editor and reexecute the file. To suppress execution use % mark in the beginning and to suppress display of result in command window put ; at the end of the line. Continue appending following in the same file.

```
%Alternate method
```

```
s=tf('s');
```

```
G2=( s^3+5*s^2+4*s+6)/( 4*s^3+7*s^2+12*s+9)
```

(b) Transfer function to pole-zero conversion, and reverse

```
[z,p,k]=tf2zp(num,den) %z =num roots, p=den roots, k=ratio of coefficient of highest  
%powered term of s in num and den that is gain of TF with normalized num and den  
%polynomial.
```

%(c) To draw the pole zero plot

pzmap(num,den)

%(d) zp2tf to zero pole to transfer function

[num1,den1]=zp2tf(z,p,k)

%(e) To find the partial fraction expansion of the transfer function

[r,p,k]=residue(num,den)% r=residue that is constant multiplier for each factor with roots p
%and k is gain of transfer function with normalization of num and den polynomial

%(f) r,p,k to transfer function

[num2,den2]=residue(r,p,k)

%(g) Representation of Transfer function of 2 o/p 1 i/p system)(2X1 matrix)

G11G21= tf({1 ; [1 2 3]} , {[1 2] ; [1 3 6 11 0]}) % specifies the two-output, one-input
%transfer function

%Result: Transfer function from output 1 to input1 G11 and output 2 to input1 G21 are
%respectively

$$\#1: \frac{1}{s+2}$$

$$\#2: \frac{s^2 + 2s + 3}{s^4 + 3s^3 + 6s^2 + 11s}$$

Problem 2

(a) Consider a controller $G_1(s) = \frac{s+1}{s+2}$ and plant $G_2(s) = \frac{1}{500s^2}$ as two system transfer functions. Obtain the overall transfer function of the system when they are connected in a) cascade b) parallel c) feedback

%(a) Cascaded system when both controller and plant are in series in the forward path

numg1=[1 1];deng1=[1 2]; numg2=[1];deng2=[500 0 0];

[nums,dens]=series(numg1,deng1,numg2,deng2)

printsys(nums,dens)

%Alternatively

s=tf('s');

G1=(s+1)/(s+2);

G2=1/(500*s^2);

GC=G1*G2

%Alternatively

[nums]=conv(numg1,numg2)

[dens]=conv(deng1,deng2)

syss=tf(nums,dens)

%(b)Parallel system when plant and controller are in parallel forward paths

```
[nump,denp]=parallel(numg1,deng1,numg2,deng2)
printsys(nump,denp)
%Alternatively
GP=G1+G2
```

%(ci)-ve unity feedback systems with G1 and G2 are in series in forward path

```
[ncl,dcl]=feedback(nums,dens,1,1,-1)
printsys(ncl,dcl)
%Alternatively
[ncl,dcl]=cloop(nums,dens)
printsys(ncl,dcl)
```

%(cii) +ve unity feedback systems with plant and controller are in series in forward path

```
[nclp,dclp]=feedback(nums,dens,1,1,+1)
printsys(nclp,dclp)
%Alternatively
[nclp,dclp]=cloop(nums,dens,+1)
printsys(nclp,dclp)
```

% (ciii) feedback system with G2 in forward path and G1 in feedback path with

%-ve feedback

```
[nclgh,dclgh]=feedback(numg2,deng2,numg1,deng1,-1)
printsys(nclgh,dclgh)
```

Exercise 1:

Consider the transfer function $G(s) = (6s^2 + 1)/(s^3 + 3s^2 + 3s + 1)$ and $H(s) = (s+1)(s+2)/(s+2i)(s-2i)(s+3)$. Perform the following (i) Compute the poles and zeros of $G(s)$ (ii) Express $H(s)$ as a ratio of two polynomials in s (iii) Obtain $G(s)/H(s)$, also find its pole zero plot.

% (i) compute the poles and zeros of G(s)

```
numg=[6 0 1];deng=[1 3 3 1];
Z=roots(numg); P=roots(deng)%Z are zeroes and P are poles , here k=6.
```

%(ii)Express H(s) as a function of two polynomials

```
n1=[1 1]; n2=[1 2]; d1=[1 2*i]; d2=[1 -2*i]; d3=[1 3];
numh=conv(n1,n2);
denh=conv(conv(d1,d2),d3);
H=tf(numh,denh)
```

%(iii)Obtain G(s)/H(s),also find its pole zero plot.

```
num=conv(numg,denh);
den=conv(deng,numh)
printsys(num,den)
pzmap(num,den)
```

Additional Exercise: Solve the following using appropriate Matlab code.

Exercise 2: Consider the two polynomials $p(s) = s^2 + 2s + 1$ and $q(s) = s + 1$, compute the following using MATLAB.

- i) $p(s)q(s)$ ii) poles and zeros of $G(s) = \frac{q(s)}{p(s)}$ iii) $p(-1)$

Exercise 3: Obtain the factored form and pole zero plot of the following transfer function

$$G(s) = \frac{s^2 + 6s + 8}{s^4 + 8s^3 + 12s^2 + 16s + 20}$$

L1a: Time domain specifications and steady state errors

Problem 1:

Obtain the unit step response of the system whose closed loop transfer function is given

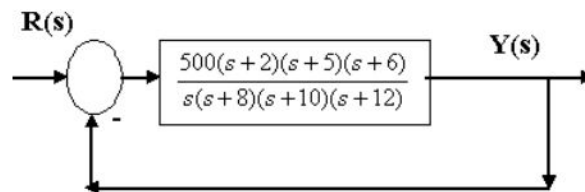
by $\frac{C(s)}{R(s)} = \frac{25}{s^2 + 4s + 25}$

Also obtain the time domain specifications i) Maximum overshoot, ii) peak time, iii) rise time, iv) delay time, v) settling time and verify result theoretically.

%step response of a system

```
num=[0 0 25]; den=[1 4 25];  
figure(1); step(num,den) %plots step response of the system  
title('unit step response of G(s)=25/s^2+4s+25')  
xlabel('t sec'); ylabel('output')  
grid on;  
%On plot right click mouse and select characteristics option to see time response  
%specifications
```

Problem 2



%Find the steady state errors

```
numg=500*poly([-2 -5 -6])  
deng=poly([0 -8 -10 -12])  
G=tf(numg,deng);
```

%check stability

```
T=feedback(G,1);  
poles=pole(T)
```

%step input

```
kp=dcgain(G)%Position(step) error coefficient  
ess=1/(1+kp)
```

%ramp input

```
numsg=conv([1 0],numg);  
sG=tf(numsg,deng)  
sG=minreal(sG)  
Kv=dcgain(sG)%Velocity error coefficient  
ess=1/Kv
```

%parabolic input

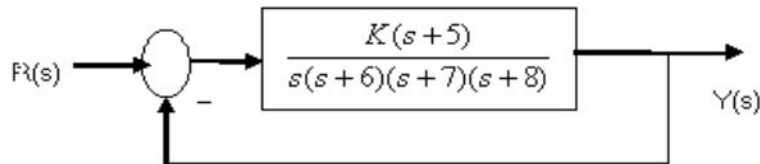
```
nums2g=conv([1 0 0],numg)  
s2G=tf(nums2g,deng)
```

```

s2G=minreal(s2G)
Ka=dcgain(s2G)%acceleration(parabolic) error coefficient
ess=1/Ka

```

Problem 3: %Gain design to meet a steady state error specification



```

%consider with K=1
numg=[1 5];
deng=poly([0 -6 -7 -8]);
G=tf(numg,deng);
numgkv=conv([1 0],numg);
GKv=tf(numgkv,deng)
GKv=minreal(GKv);
Kvi=dcgain(GKv)
ess=0.1%desired steady state velocity error
K=1/(ess*Kvi)
T=feedback(K*G,1)
poles=pole(T)

```

Exercises:

Exercise 1:

Obtain the impulse response of the closed loop transfer function $\frac{C(s)}{R(s)} = \frac{5s^2 + 3s + 6}{s^3 + 6s^2 + 11s + 6}$.

% Impulse response

```

num=[5 3 6];
den=[1 6 11 6];
figure(1)
impz(num,den)% impulse response of original system
grid on; title('unit impulse response')
xlabel('t')
ylabel('output')

```

Exercise 2:

Obtain the unit step responses of the following systems whose closed loop transfer function are given by $\frac{C(s)}{R(s)} = \frac{1}{s^2 + 0.5s + 1}$ and $\frac{C(s)}{R(s)} = \frac{1}{s^2 + 0.5s + 4}$. Plot their step responses curves in one figure window.

%program

```

clc; figure(1)
t=0:0.1:20

```

```

num1=[0 0 1]; den1=[1 0.5 1]; num2=[0 0 1]; den2=[1 0.5 4];
[y1,x1,t]=step(num1,den1,t)
[y2,x2,t]=step(num2,den2,t)
plot(t,y1,'g',t,y2,'r')
xlabel('t(s)')
ylabel('outputs y1 and y2')
title('Step response of two systems')
legend('y1', 'y2')

```

Exercise 3: For the following closed loop transfer functions obtain the step/impulse/ramp responses of the given system. ii) And also obtain the time domain specification from step response and verify result theoretically.

$$\frac{C(s)}{R(s)} = \frac{\omega_n^2}{s^2 + 2\xi\omega_n s + \omega_n^2}, \text{ where } \xi = 0.6 \text{ and } \omega_n = 8.$$

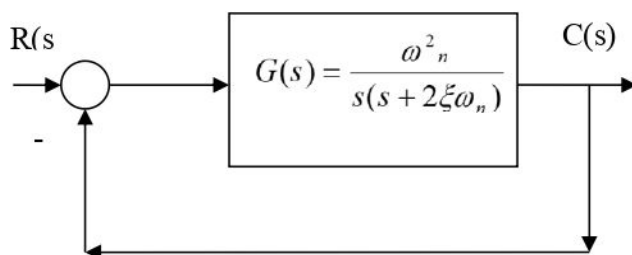
Exercise 4: Determine the step, ramp, and parabolic error constants and the steady state error for the unity negative feedback control system with following open loop transfer function. Also plot the respective time domain responses.

$$G(s) = \frac{5s^2 + 2s + 1}{s(s+2)(s+8)(s+45)}$$

L1b: Frequency response and Frequency domain analysis

Aim: To determine the frequency response of a second-order system and evaluate the frequency domain specifications.

Theory:



Consider the unity negative feedback closed loop system having open loop transfer function

$$G(s) = \frac{\omega_n^2}{s(s + 2\xi\omega_n)}.$$

Choose appropriate value of ω_n and ξ . Choose ξ to be < 0.707 . Obtain the closed loop

transfer function $\frac{C(s)}{R(s)} = \frac{G(s)}{1 + G(s)} = \frac{\omega_n^2}{s^2 + 2\xi\omega_n s + \omega_n^2}$. This is a typical second order system.

Problem 1:

To obtain the frequency response of the above system and to obtain the frequency response specifications, following Matlab Code may be used.

%Matlab Code:

```
clear all
close all
clc
wn=input('enter the natural frequency of oscillation wn=');
zeta=input('enter the damping ratio zeta='); %zeta <0.707
n=wn^2;
d=[1 2*zeta*wn wn^2];
sys=tf(n,d); %given closed loop second order transfer function is defined
w=logspace(-2,3,100); % frequency range between 0.01 & 1000 with 100 logarithmically
equally spaced points.
h=freqresp(sys,w); %evaluates complex value of system at every frequency point
reh=real(h(:)); %extracts real part at every frequency
imh=imag(h(:)); %extracts imaginary part at every frequency
magh=sqrt(reh.^2+imh.^2); %magnitude in absolute value
```

```
%Alternatively
mh=abs(h)%
maga=mh(:); %maga=maga
magdb=20*log10(magh);
phh=atan2(imh,reh); %quadrantwise angle computed in radians
%alternatively
ph=angle(h);
ph2=ph(:); %ph2=phh
phd=phh*180/pi;
C=[w',magdb,phd];
display(C)
subplot(2,1,1),semilogx(w,magdb);
grid on,
zoom on
subplot(2,1,2),semilogx(w,phd);
grid on,
zoom on
```

%To read the value of resonant frequency and resonant peak from plot use

```
[wr,mr]=ginput(1)
```

%To read bandwidth

```
[wb,mb]=ginput(1)
```

%theoretical Verification

```
%Mr= 20*log10( $\frac{1}{2\xi\sqrt{1-\xi^2}}$ );
```

```
%  $\omega_r = \omega_n \sqrt{1-2\xi^2}$  ;
```

```
%  $\omega_b = \omega_n * \sqrt{(1-2*\xi^2) + \sqrt{4*\xi^4 - 4*\xi^2 + 2}}$ 
```

Result: From the plot obtain the resonance peak M_r , resonance frequency ω_r and the bandwidth. Compare these values with the theoretically obtained values.

Resonance Peak= M_r = db

Resonance frequency= ω_r = rad/sec

Bandwidth= ω_b = rad/sec

Exercise 1:

Using Matlab command find the magnitude and phase angle of the system with transfer

function $G(s) = \frac{5}{s+2}$ at $\omega = 3\text{rad/sec}$

%Matlab code

GNUM=[5];GDEN=[1 2]; s=3*j;

Gj3=polyval(GNUM,s)/polyval(GDEN,s) %complex function evaluation at a given complex
%value

magGj3=abs(Gj3)

phaseGj3=angle(Gj3)*180/pi

Exercise 2

Consider the standard second order transfer function $M(s) = \frac{\omega_n^2}{s^2 + 2\xi\omega_n s + \omega_n^2}$. By

defining the normalized frequency $\omega_v = \frac{\omega}{\omega_n}$, plot the frequency response for various

values of $\xi = 0.25, 0.5, 0.707, 1$.

%Matlab Code

wv=0:0.05:3;

z=[0.25 0.5 0.707 1];

for k=1:4

 mnum=[0 0 1];mden=[1 2*z(k) 1];

 mjomega=freqs(mnum,mden,wv);

 mmag=abs(mjomega);

 plot(wv,mmag)

 gtext('z(k)')

 title('frequency response of M(S)')

 xlabel('omegav');

 ylabel('|M(jwv)|');

 hold on

end

gtext('\zeta=0.25')%click on the plot to label plot with zeta=0.25

gtext('\zeta=0.5')

gtext('\zeta=0.707')

gtext('\zeta=1')

grid;

L1c: STABILITY ANALYSIS

Theory: For a minimum phase system, both gain margin and phase margin should be positive for closed loop stability.

Nyquist stability criteria:

Using Polar plot: $-1+j0$ point in GH plane is not enclosed by polar plot, then the closed loop system is stable.

Using Nyquist plot: If the minimum phase system has no open loop poles on RHS of s-plane, then the number of encirclements of $-1+j0$ point should be zero.

If the minimum phase open loop system has p number of open loop poles on RHS of s-plane, then the number of encirclements of $-1+j0$ point in GH plane should be p in clock wise direction (+). That is, $z=p-n=0$, implying there is no RHS zero of characteristic equation in RHS of s plane. Else, that is if p is not equal to zero and $n=0$, or $n < p$, or if $p=0$ and n is in counter clock wise direction, the closed loop system is unstable.

Using Root locus: The closed loop system is stable for a range of open loop gain, for which the closed loop poles all lie on LHS of s-plane. The value of K corresponding to imaginary axis crossing of root locus, gives marginal stability and corresponding imaginary axis crossing frequency, gives frequency of sustained oscillation. If K is set above this critical value, closed loop system becomes unstable.

Problem 1

Determine the stability of the negative unity feedback system whose open loop transfer function is given by $G(s) = \frac{1.25}{s(s+1)(0.5s+1)}$

%Stability analysis

```
a=[1.25]; b=[1 1 0]; c=[0.5 1];  
d=conv(b,c)  
figure(1); bode(a,d)  
grid on;  
margin(a,d)  
figure(2); nyquist(a,d)  
grid on;  
figure(3); rlocus(a,d)  
grid on;
```

Result: Using Bode plot:

Gain cross over frequency

Phase crossover frequency

Gain at phase cross over frequency

Gain margin

Phase at Gain cross over frequency

Phase margin:

Comment on closed loop stability...

Using Nyquist plot:

Using Root locus:

Break away point:

Gain at Break away point

Imaginary axis crossing point= Frequency of sustained oscillation=

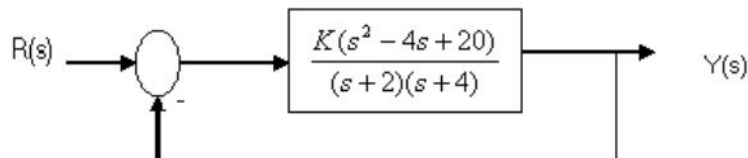
Gain at imaginary axis crossing point=

Comment on stability and relative stability:

Problem 2

Sketch the root locus for the system shown in figure and find the following

- The exact point and gain where the locus crosses the 0.45 damping ratio line.
- The exact point and gain where the locus crosses the imaginary axis.
- The breakaway point on the real axis.
- The range of K within which the system is stable.



```
numgh=[1 -4 20];
dengh=poly([-2 -4]);
GH=tf(numgh,dengh)
figure(1); rlocus(GH)
z=0.2:0.05:0.5;
wn=0:1:10
sgrid(z,wn)
figure(2); rlocus(GH)
axis([-3 1 -4 4])
z=0.45;wn=0;sgrid(z,wn)
for k=1:3
[k,p]=rlocfind(GH)
end
```

Exercises:

Exercise 1: Draw Bode diagram for TF of $G(s) = \frac{e^{-2s}}{s(s+1)}$ and determine the stability

```
nm=30; a=-1;b=1;
w=logspace(a,b,nm)
Gnum=[0 0 1];Gden=[1 1 0];
[mag,phase,w]=bode(Gnum,Gden,w);
db=20*log10(mag);
phased1=(-2)*57.296*w;
phase=phase+phased1;
subplot(211), semilogx(w,db)
title('BODE DIAGRAM')
```

```

xlabel('frequency');
ylabel('magnitude (dB)');grid on;
subplot(212),semilogx(w,phase)
xlabel('frequency');ylabel('phase (degrees)');grid on;

```

Exercise 2:

Obtain the polar and Nyquist plot of the system having its open loop transfer function $G(s)H(s) = \frac{k}{(s)(s+0.5)(s+2)}$; take $k=1, 10$. Also determine the gain cross over frequency, phase margin, phase cross over frequency and gain margin. Verify the Nyquist theorem and validate the result by determining the roots of the characteristic equation using MATLAB.

```

%to obtain polar/Nyquist plot
close all; clear all;
k=input('enter the value of open loop gain k=')
pol=input('enter three open loop poles [p1,p2,p3]')
n=k; d=poly(pol);
sol=tf(n,d); %open loop transfer function
[re,im,w]=nyquist(sol);
mag=sqrt(re(:).^2+im(:).^2)
magdb=20*log10(mag);
phd=(180/pi)*atan2(im(:),re(:));
c=[w,magdb,phd];
display(c)
plot(re(:),im(:));%plots polar plot
figure(2), plot(re(:), im(:), re(:), -im(:));%plots Nyquist plot
[x,y]=scircle1(0,0,1);
hold on, plot(x,y) %plots unit circle
%to plot Nyquist plot
figure(3), nyquist(n,d)
oltf=tf(n,d);
cltf=feedback(oltf,1); %to get closed loop transfer function
[ncl,dcl]=tfdata(cltf,1);
% clpoles=roots(dcl);
clpoles=pole(cltf)

```

Specimen Calculation: Using $[regc,imgc]=ginplot(1)$ and $[repc,impc]=ginplot(1)$ on command window, calculate phase angle and gain at gain cross over and phase cross over frequencies and hence the margins. Or using the matrix 'c' frequency at cross over and gain and phase at respective points can be calculated. Note: If the angle is positive at gain cross over frequency, subtract 360 from displayed angle to obtain actual angle.

Additional Exercises:

Exercise 3: Obtain the Nyquist plot and determine the stability of the following unity feedback systems with open loop transfer functions are i) $G(s) = \frac{15(s+5)}{s(s+2)(s^2+6s+15)}$

ii) $G(s) = \frac{12(s+2)}{s^2-6s+10}$

Exercise 4: Obtain the root locus plot for $K>0$, the following unity feedback systems with open loop transfer functions are i) $G(s) = \frac{K}{s(s+4)(s+8)}$ ii) $G(s) = \frac{K(s+7)}{(s+2)(s^2+2s+3)}$. Also find the range of K for which the systems are stable. Also find the break-away points.

L2a: State space analysis

Problem 1

Aim: To represent transfer function in state space form and vice versa. Also to obtain step response of SISO system

Consider a plant transfer function $G(s) = \frac{s^3 + 5s^2 + 4s + 6}{4s^3 + 7s^2 + 12s + 9}$

% transfer function to state space conversion

```
num1=[1 5 4 6];  
den1=[4 7 12 9];  
[A,B,C,D]=tf2ss(num1,den1)
```

%state space to transfer function

```
[num3,den3]=ss2tf(A,B,C,D)  
%step response from state space model  
figure(1)  
step(A,B,C,D)
```

Problem 2: MIMO system time domain response

```
clear all;  
%To Obtain the transfer function of a MIMO (2 o/p, 2 i/p) system 2X2 matrix  
A=[0 1;-25 -4]; B=[1 1;0 1]; C=[1 0;0 1]; D=[0 0;0 0];  
[num1,den1]=ss2tf(A,B,C,D,1)% gives y1(s)/u1(s) and y2(s)/u1(s) with u2 zero  
[num2,den2]=ss2tf(A,B,C,D,2)%Gives y1(s)/u2(s) and y2(s)/u2(s) with u1 zero
```

% To find the eigen value

```
eig(A)% Roots of denominator polynomial of transfer function. All -ve system stable.
```

%Step Response curves of a MIMO system.

```
figure(2)  
[y1 y2 t]=step(A,B,C,D,1);  
plot(t,y1,t,y2)  
title('step Response plots: Input=u1 (u2=0)')  
text(3.4,-0.06,'Y1')  
text(3.4,1.4,'Y2')  
figure(3)  
[y1 y2 t]=step(A,B,C,D,2);  
plot(t,y1,t,y2)  
grid  
title('step Response plots: Input=u2 (u1=0)')  
text(3,0.14,'y1')  
text(2.8,1.1,'y2')
```

Problem 3:

Diagonalization of A matrix in Controllable canonical form

A linear time invariant system is described by the following constant matrices

```
A=[0 1 0;0 0 1;-6 -11 -6];
```

B=[0;0;2];C=[1 0 0];

D=[0];

Determine the following using matlab commands.

- (i) Transform the above state model in to Jordan canonical form $\hat{A}, \hat{B}, \hat{C}, \hat{D}$
- (ii) Show that eigen values of A are equal to $\hat{A}$
- (iii) Determinant of A is equal to determinant of $\hat{A}$
- (iv) The trace of A is equal to trace of $\hat{A}$
- (v) The transfer functions of A,B,C,D = transfer functions of $\hat{A}, \hat{B}, \hat{C}, \hat{D}$

MATLAB Code:

```
L=eig(A)
P=[1 1 1;L(1) L(2) L(3);L(1)^2 L(2)^2 L(3)^2];%Van der monde transformation matrix for
%transformation of A in controllable canonical form, to diagonal form for distinct root case
PI=inv(P);
ASTILDA=PI*A*P
BSTILDA=PI*B
CSTILDA=C*P
DSTILDA=D
L1=eig(ASTILDA)
DETA=det(A)
detstilda=det(ASTILDA)
TRA=trace(A)
TRASTILDA=trace(ASTILDA)
[num den]=ss2tf(A,B,C,D)
S1=tf(num,den)
[num1 den1]=ss2tf(ASTILDA,BSTILDA,CSTILDA,DSTILDA)
S2=tf(num1,den1)
```

L2b: Controllability, observability and Pole placement

Problem 1:

To Check the controllability and observability of the control system.

A=[0 1 0 ;0 0 1;-6 -11 -6];

B=[0;0;1];C=[4 5 1];

Ob = obsv(A,C)

no=size(A);

if rank(Ob)==min(no)

display('system is observable')

else

display('system is not observable')

end

CO = ctrb(A,B)

if rank(CO)==min(no)

display('system is controllable')

else

display('system is not controllable')

end

CO1=[B A*B A^2*B];%Manual method to compute the Controllability matrix for 3 state system

```
rank(CO1) % checks the rank of controllability matrix
OB1=[C; C*A; C*A^2];%Manual method to compute the Observability matrix for 3 state
system
rank(OB1) % checks the rank of observability matrix
```

Problem 2

%(1) Pole placement by Ackermann's Formula

Theory: For a system described by state model, A, B, C, D, and (A,B) pair is controllable, then arbitrary placement of closed loop poles can be done by a state feedback control law $u = -Kx$, given $K = [0 \ 0 \ 1] * \text{inv}(M) * \phi(A)$, where M or CO is the controllability matrix, $\phi(A)$ is the matrix polynomial of desired characteristic equation. (no of elements in K equals n, order of system which all except last entry zero and last entry is 1)

%Illustrative example 1

```
clear all;
A=[0 1 0 ;0 0 1;-6 -11 -6 ];
B=[0;0;1];C=[4 5 1]; % Third order system example
eig(A)%poles of open loop system A B C D
CO=[B A*B A^2*B];%Controllability matrix
rank(CO);
nc=size(A);
if rank(CO)==nc
display('system is controllable and pole placement is possible')
else
display('system is not controllable and pole placement is not possible')
exit
end
P=[-8.4563 0 0; 0 -0.7718+0.5919*i 0; -0.7718-0.5919*i 0 0]%Canonical representation with
desired closed loop poles in matrix form
P1=[-8.4563; -0.7718+0.5919*i; -0.7718-0.5919*i]%Canonical representation with desired
closed loop poles in vector form
poly(P)%desired characteristic polynomial coefficients
Phi=polyvalm(poly(P),A);%desired characteristic polynomial formed with matrix A;
%A^3+alpha1*A^2+alpha2*A+alpha3
K0=[0 0 1]*(inv(CO))*Phi%Ackerman's formula
Acl0=A-B*K0
eig(Acl0)%poles of closed loop system with u=-Kx.

% Verification
Kacker1=acker(A,B,P1)
Acl1=A-B* Kacker1
eig(Acl1)

Kacker2=place(A,B,P1)
Acl2=A-B*Kacker2
eig(Acl2)
```

Exercise 1a:

%(2) pole placement by controllable canonical form transformation method 1

%Note: System order considered as 3, for change in dimension of A matrix(system order), %appropriate change need to be made in ai's, alphai's,w matrix,j vector, k vector

```
A=[0 1 0 ;0 0 1;-6 -11 -6 ];
B=[0;0;1];C=[4 5 1]; % Third order system example
eig(A)%open loop poles
no=size(A);
Co=ctrb(A,B);
rank(CO)
if rank(CO)==min(no)
display('system is controllable and arbitray pole placement can be done')
else
display('system is not controllable and arbitrary pole placement is not possible')
end
ja=poly(A)%open loop system characteristic polynomial
a1=ja(2);a2=ja(3);a3=ja(4);%extraction of characteristic equation polynomial
w=[a2 a1 1;a1 1 0;1 0 0];%weighing matrix
t=CO*w%transformation matrix to convert A to controllable canonical form with last row as
%characteristic equation coefficients with negative sign in reverse order
j=[-1+i*2 0 0;0 -1-i*2 0;0 0 -6];%desired pole location for closed loop system with state
%feedback u=-kx
jj=poly(j);%desired characteristic polynomial
alpha1=jj(2);alpha2=jj(3);alpha3=jj(4);%extraction of closed loop characteristic polynomial
%coefficients
k=[alpha3-a3 alpha2-a2 alpha1-a1]*inv(t)%feedback gain matrix
Acl=A-B*k;
eig(Acl)%final closed loop poles
```

Exercise 1b:

%(3)Pole placement by controllable canonical transformation method 2

```
A=[0 1 0 ;0 0 1;-6 -11 -6 ];
B=[0;0;1];C=[4 5 1]; % Third order system example
eig(A)%open loop poles
no=size(A);
CO=ctrb(A,B);
rank(CO)
if rank(CO)==min(no)
display('system is controllable and arbitray pole placement can be done')
else
display('system is not controllable and arbitrary pole placement is not possible')
end
ja=poly(A)%open loop system characteristic polynomial
a1=ja(2);a2=ja(3);a3=ja(4);%extraction of characteristic equation polynomial
invcon=inv(CO)
p1=invcon(3,:)%Last row of inverse of controllability matrix extracted

V=[p1;p1*A;p1*A*A]% Transformation matrix to transform A to controllable canonical
form
```

```

AC=V*A*inv(V)%controllable canonical form transformation of A
BC=V*B;
alpha=AC(3,:)%Extraction of last row of Controllable canonical form of A
P=[-1+i*2 -1-i*2 -6]%Desired closed loop poles vector
lambda=poly(P)%Desired characteristic polynomial coefficients
lambda=lambda(1,2:4)%extraction of coefficients except first one.
K1=[alpha(1)+lambda(3) alpha(2)+lambda(2) alpha(3)+lambda(1)]%SFB gain matrix of A in
%CCF
%If A itself is controllable comment following line
K=K1*V%SFB gain matrix for A
eig(A-B*K)
eig(AC-BC*K1)
%Alternate method by using Ackerman formula
Kacker=[0 0 1]*inv(Co)*[A*A*A+lambda(1)*A*A+lambda(2)*A+lambda(3)*eye(3)]
%Alternate method using Acker command
Kacker1=acker(A,B,P)

```

Exercise 2:

Using place command

Illustrative example 2: Find the control gain F such that when for the system,

$$A = \begin{bmatrix} -1 & 0 & 0 \\ 1 & -2 & 0 \\ 2 & 1 & -3 \end{bmatrix} ; \quad B = \begin{bmatrix} 10 \\ 1 \\ 0 \end{bmatrix}$$

is controlled by $u(t) = -Fx(t)$, the closed loop poles are at $s = [-1+i*2 \ -1-i*2 \ -6]$, verify the control gain by finding the eigen values of $A-BF$.

Program:

```

% Pole placement design.
A=[-1 0 0;1 -2 0;2 1 -3];B=[10;1;0];
P=[-1+i*2 -1-i*2 -6]% Vector of desired closed loop poles
F=place(A,B,P)%Feedback gain matrix, u=-Fx
A_cl=A-B*F
eig(A_cl)%Verification of closed loop poles

```

Exercise 3:

For the state model given below obtain the state models (i)) Jordan canonical form (ii) Plot the pole zero plot of the system.

$$\dot{X} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -2 & 1 & -3 \end{bmatrix} X + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} U ; \quad Y = [4 \ 1 \ 0]$$

Determine system controllability and observability.

$$A = \begin{bmatrix} -2 & -2.5 & -0.5 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} ; \quad B = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} ; \quad C = [1 \quad 4 \quad 3.5]$$

Place the poles of the system in illustrative example 2 at $s=-4, -8, -10$.

Exercise 4:

Place the poles of the system.

$$A = \begin{bmatrix} -0.1 & 5 & 0.1 \\ -5 & -0.1 & 5 \\ 0 & 0 & -10 \end{bmatrix} ; \quad B = \begin{bmatrix} 0 \\ 0 \\ 10 \end{bmatrix} ;$$

at (i) $s = -1+j5, -1-j5$ and -10 (ii) $-5, -60$ and -70 , find and compare the norms of F .

Exercise 5:

A SISO system represented by the state equation

$$A = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -2 & -2 \\ -1 & 0 & -3 \end{bmatrix} \text{ and } B \text{ matrix is given by 2 cases } B1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \text{ and } B2 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$

For each B i) determine controllability

- ii) Find eigen values
- iii) Find transformation matrix V that transforms the state equation to controllable canonical form
- iv) Determine state feedback matrix K required to assign eigen values to $\{-2 - 4 - 6\}$

L3: Proportional controller design, lead and lag compensator design

Problem 1:

Proportional controller (Gain adjustment) design using Bode plot:

For the control system shown in figure (a), find the value of gain K to yield 9.5% overshoot in the transient response for a step input.

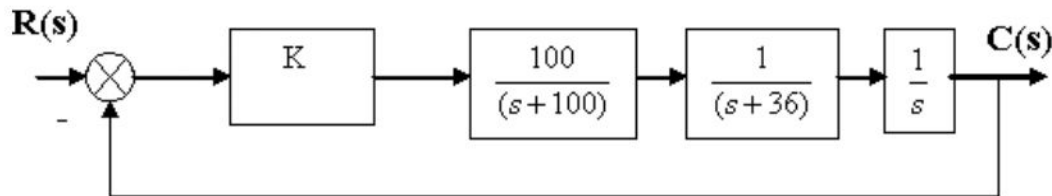


Figure (a)

Procedure

For the given overshoot, find the required damping ratio and hence obtain the required phase margin.

%design1

%Gain adjustment; enter the specifications

OS=9.5; numg=[100]; deng=poly([0 -36 -100]);

G=tf(numg,deng);

%TO FIND REQUIRED DAMPING RATIO

z=(-log(OS/100))/(sqrt(pi^2+log(OS/100)^2));

%To find the required phase margin

pmreq=atan(2*z/(sqrt(-2*z^2+sqrt(1+4*z^4))))*(180/pi);

- **Find the phase margin of the given system**

w=logspace(-2,3,1000);

bode(G,w);

grid on;

- **Find the additional gain needed to produce the required phase margin**

Calculation: After obtaining Bode plot perform following calculation.

From the plot determine the frequency at which desired phase margin angle is satisfied.

That is mark w_{gc} as that frequency where phase angle is -180+pmreq

Determine the gain at this frequency from the graph. Let it be M db.

To make this as gain cross over frequency the magnitude plot should cross zero dB at this frequency.

Hence gain required is $K=10^{(-M/20)}$

- **Verify the result from closed loop step response.**

% verify the design by plotting

numgn=K*numg

hold on

bode(numgn,deng)

%To verify closed loop step response with designed K

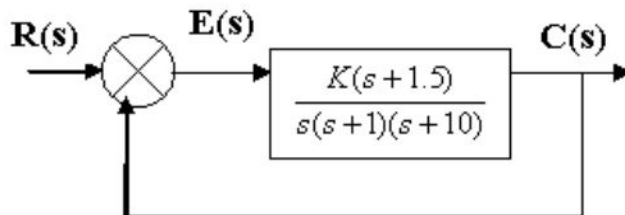
T=feedback(K*G,1);

```
Figure();
step(T)
CGC=K*G;
```

Exercise 1:

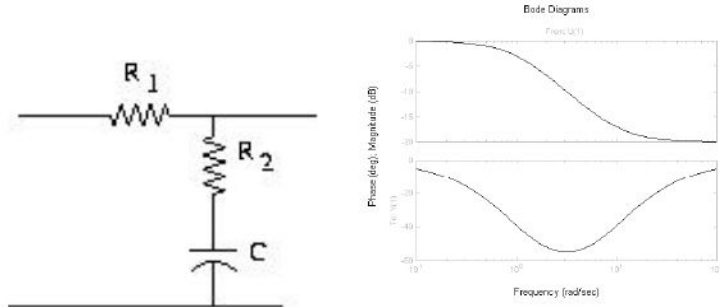
Using Root locus

For the system shown below, design the value of gain K to yield 1.52% overshoot. Also estimate the settling time, peak time and steady state error.



```
%design 2
% Root locus method (DO NOT TYPE THE FULL PROGRAM)
%For the system given design the value of gain, to yeild 1.52%overshoot.
%also estimate settling time, peak time and steady state error.
%Enter the system transfer function
clear;clc;clf;
num=[1 1.5];
den=poly([0 -1 -10]);
G=tf(num,den);
%TO OBTAIN THE ROOT LOCUS PLOT
rlocus(G);
title('original root locus plot');
pause
%enter the desired %overshoot
OS=1.52;
%obtain the damping ratio
z=(-log(OS/100))/(sqrt(pi^2+log(OS/100)^2));
%to mark desired damping ratio
sgrid(z,0);
title([' root locus plot with ',num2str(OS),'%overshoot line']);
%to obtain gain K and closed loop poles for point selected, (zoom the figure and
%observe step by step.
[K,P]=rlocfind(G);
%obtain closed loop transfer function for the given K
T=feedback(K*G,1);
pause
step(T);
%verify the time domain specification using graph and equations
TS=-4/real(P(1))
TP=pi/imag(P(1));
s=tf([1 0],[0 1]);
KV=dcgain(s*G);
```

THEORY: LAG compensator design using Frequency response:



$$TF = \frac{1}{\beta} * \frac{\left(s + \frac{1}{\tau}\right)}{\left(s + \frac{1}{\beta\tau}\right)} ; \tau = R_2 C ; \beta = (R_1 + R_2) / R_2$$

NOTE: This form useful for root locus based design, where amplifier gain can be used to to cancel $\frac{1}{\beta}$, and get desired steady state error requirement met by multiplying K to meet desired pole location with β to meet the steady state error requirements.

High frequency attenuation = $20 * \log_{10} (1/\beta)$ db

For Frequency domain design the form $\frac{\tau s + 1}{\beta \tau s + 1}$ is useful.

Lag Compensator design:

Design a Lag compensator for a given OLTf for achieving a specified phase margin of 30 degrees.

Problem 2:

For a unity feedback control system with $G(s) = \frac{1}{s(s+1)(0.5s+1)}$, design a phase lag compensator such that the closed loop system will satisfy the following requirements. $K_v=5/s$, Phase margin= 40° , gain margin >10 db.

Procedure:

1. Bode plot of the uncompensated system is plotted after setting the value of the gain constant from the steady state requirement.

%lag compensator design using Bode plots

%enter the design specifications

```
KV=5;PM=40;GM=10;
%ENTER THE TRANSFER FUNCTION OF THE UN COMPENSATED SYSTEM
numg=[1];deng=conv(conv([1 0],[1 1]),[0.5 1]);
G=tf(numg,deng);
%TO FIND k, to meet steady state error criteria, use sG(s)H(s)
K=KV/(dcgain(conv([1 0],numg),deng));
```

2. Gain margin and Phase margin of Gain adjusted uncompensated system is measured.

```
w=logspace(-1,2,100);
[mag,ang]=bode(K*numg,deng,w);
[gm,pm,wcg,wcp]=margin(mag,ang,w);
```

Calculation:

3. The frequency corresponding to the required phase margin+allowance (~8 to 12 deg (to cancel additional phase lag due to lag network at new gain cross over frequency) is determined from the Phase curve. This gives ω_m . That is, from the Bode plot, identify the frequency ω_m as where phase angle is $-180+PM+allowance$.

4. To achieve the specified phase margin at this frequency, the gain curve should pass through 0 db at this frequency. This is achieved by setting the maximum attenuation of the lag-compensating network to be equal to the gain of the uncompensated system at this frequency. i.e., gain of the uncompensated system at new Gain Cross over frequency ω_m , M dB is measured from the magnitude plot and is set equal to $-20*\log(1/\beta)$. Thus, $\beta > 1$ is determined.

That is Measure gain in dB at ω_m from magnitude plot. Compute β using relation

$20 \log_{10} \beta = M$ and $\beta = 10^{(M/20)}$.

5. To minimize the phase contribution of LAG network around the new gain cross over frequency, the upper corner frequency of the LAG compensating network is chosen to be at least 1 decade less than new gain crossover. Hence the TF of the LAG compensating

$$\text{network given by } G_c(s) = \frac{1}{\beta} \frac{\left(s + \frac{1}{\tau}\right)}{\left(s + \frac{1}{\beta\tau}\right)} = \frac{(\tau s + 1)}{(\beta\tau s + 1)}$$

is designed as, $1/\tau < \omega_m/10$

Hence, the lower corner frequency of lag network is $1/\beta\tau$.

6. Obtain the overall TF using “series” command in MATLAB. Obtain the Bode plot of the overall TF. Hence determine the new gain and phase margin.

%Lag compensator transfer function:

```
wh= $\omega_m$ /10; w1=wh/ $\beta$ ; kc=w1/wh;
```

```
lagc=tf(kc*[1 wh],[1 w1]);
```

%compensated system

```
CGCS=K*G*lagc;
```

```
S=tf([1 0],[0 1]);
```

```
KVF=dcgain(S*CGCS);
```

```
CLS=feedback(CGCS,1);
```

```
margin(CGCS); pause; step(CLS)
```

Result: Observe and plot the Bode plot of uncompensated with gain compensated system, compensated system, and closed loop step response. Verify the performance.

LAG Compensator using Root locus:

Gain of the system is increased as much as possible without appreciably changing root locus in the vicinity of dominant poles of the closed loop system. This is achieved by restricting the angle contribution of lag network to be negligible say less than 5 deg. Hence the poles and zeros of lag network are placed close to each other and nearer to the origin. The pole and zero of lag compensator is determined as follows:

Open loop poles and zeros of uncompensated system are located in s plane and the dominant closed loop poles are identified to satisfy the given transient response specification. This point should satisfy and phase angle condition of root locus as well as magnitude condition. Applying magnitude condition the open loop gain of uncompensated system is determined as K_u at the dominant pole. Now with the specified steady state requirements, the required open loop gain K to meet the error requirement is determined. The lag network to be introduced in

cascade in the forward path has transfer function $G_c(s) = \frac{1}{\beta} \frac{(s + \frac{1}{\tau})}{(s + \frac{1}{\beta\tau})}$. With the

introduction of the network the overall forward path gain now becomes $K_u\beta$, after using an amplifier with gain β . To meet the steady state error requirement, this should be equal to K .

Hence β is determined as $\frac{K}{K_u}$. However to compensate for the change in magnitude condition

due to the introduction of compensator, this gain is compensated by a multiplying with a tolerance value of 1.1. The compensator zero $-\frac{1}{\tau}$ is placed at about 0.1 of second real pole

from origin of uncompensated system. With this zero $-\frac{1}{\tau}$ and the value of β determined, the

pole can be determined. With this the open loop transfer function of compensated system is written as $1.1K\beta G_c G$ and overall root locus is plotted and the specifications are verified. If not satisfied, the procedure is repeated with slightly modifying the position of zero and pole of compensator.

Problem 3:

For a unity feedback control system with $G(s) = \frac{1}{(s+1)(s+2)(s+10)}$, design a lag

compensator to improve the steady state error by a factor of 10 if the system is operating with a damping ratio of 0.174.

Program

```
clear all; close all; clc
s=tf('s')
g1=1/((s+1)*(s+2)*(s+10))
rlocus(g1)
```

```

%design a lag compensator to improve steady state error by a factor of 10
z=0.174;
sgrid(z,0); %Marks on rootlocus  $\{\zeta\}=0.174$  line
[k,p]=rlocfind(g1)%Place cursor on intersection point of  $\{\zeta\}$  line and rootlocus and
%measure gain and poles corresponding to desired damping ratio
%for type 0 systems
kp=dcgain(k*g1)% Find position error coefficient with gain corresponding to desired DR
ess=1/(1+kp);%steady state for type 0 unity feedback ol system
%to find beta which reduces steady state error by 10
kp1=9+10*kp; %desired position error coefficient to meet specification
beta=(kp1/kp)*1.1;%HF attenuation to be provided by lag network to meet desired spec
%without changing DR
%zero of lag network placed close to origin or about 1/10 of second dominant real pole
zlag=0.1;
plag=zlag/beta;
gclag=(s+zlag)/(s+plag);%Lag network transfer function used with UF OLTF
cgc= g1*gclag%effective UF OLTF
zpk(cgc)
rlocus(cgc)
sgrid(z,0)
[k1,p]=rlocfind(cgc)%Find gain to meet desired DR by placing cursor at intersection
%type 0 systems
kpf=dcgain(k1*cgc)
ess1=1/(1+kpf)
ess/ess1
%Comparison of closed loop response of uncompensated and compensated system
cltfu=feedback(k*g1,1)
step(cltfu)
hold on
cltfc=feedback(kpf*cgc,1)
step(cltfc)

```

Exercise 2:

Lag compensator design using Root locus to get desired velocity error coefficient as $k_v=5$, $z=0.5$ and settling time as 10s.

```

clear all; close all; clc
s=tf('s')
%type 1 system
g1=1/(s*(s+1)*(s+4))
rlocus(g1)
z=0.5;TS=10;kvf=5
wn=4/(z*TS);
s1=-z*wn+j*wn*sqrt(1-z*z);%desired closed loop dominant poles
[k,p]=rlocfind(g1,s1)%finding gain to meet desired poles
%static velocity error constant
d1=s
kv=dcgain(d1*k*g1)%type 1 system
ess=1/kv;
%desired gain
k=kvf/kv;% gain to be provided by lag network to meet steady state error

```

```

beta=k*1.1;
zlag=0.2;
plag=zlag/beta;
gclag=(s+zlag)/(s+plag);
cgc=g1*gclag
zpk(cgc)
rlocus(cgc)
[k1,p1]=rlocfind(cgc,s1)%gain of compensated system which places roots and meet SS error
kvf=dcgain(d1*cgc*k1)% compare with desired value

%Comparison of closed loop response of uncompensated and compensated system
cltfu=feedback(k*g1,1)
figure(2)
step(cltfu)
hold on
cltfc=feedback(k1*cgc,1)
step(cltfc)

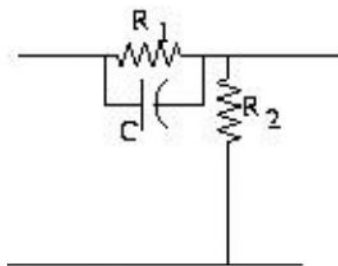
```

Exercise 3:

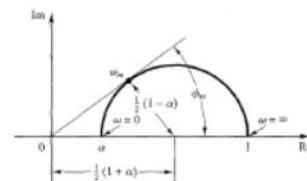
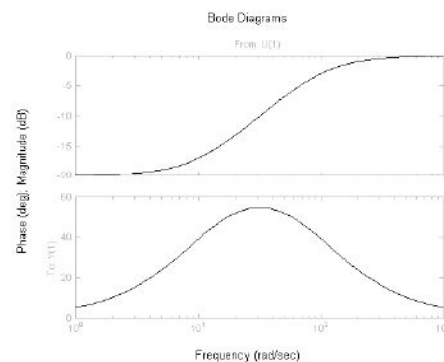
For a unity feedback control system with $G(s) = \frac{1}{(s+1)(s+2)(s+10)}$, design a lag compensator to improve the steady state error by a factor of 10 if the system is operating with a damping ratio of 0.174 and settling time 1s.

L3b: LEAD compensator design

Theory of Lead compensator design using frequency response.



$$TF = \frac{\left(s + \frac{1}{\tau}\right)}{\left(s + \frac{1}{\alpha\tau}\right)}; \quad \tau = R_1 C; \quad \alpha = R_2 / (R_1 + R_2)$$



This network is having low frequency attenuation property. That is low frequency gain α is less than one, $(20\log_{10}(\alpha)\text{db})$, which is negative). Hence in the design of compensator an

amplifier with a gain of $\frac{1}{\alpha}$ is to be cascaded with the lead network in the frequency domain design. That is effectively using lead network transfer function as $\frac{(\tau s + 1)}{(\alpha \tau s + 1)}$.

The design value of α is obtained from the maximum phase lead provided by the network. From the polar plot of lead network shown in figure, by drawing a tangent to the polar plot origin, we get the maximum phase lead angle ϕ_m provided by the network. By drawing the normal to this to the center of the semicircle, we can relate maximum phase lead angle ϕ_m to α as $\sin \phi_m = \frac{1 - \alpha}{1 + \alpha}$. Using the phase angle expression $\phi_m = \text{atan}(\omega_m \tau) - \text{atan}(\alpha \omega_m \tau)$ at

frequency ω_m , and relationship for $\tan \phi_m = \frac{1 - \alpha}{2\sqrt{\alpha}}$, from the polar plot, we get $\omega_m = \frac{1}{\tau\sqrt{\alpha}}$

From magnitude condition, the magnitude of lead network at $\omega_m = 10\log_{10}(\alpha)\text{db}$ (negative) (or $\sqrt{\alpha}$ in abs). After cascading the amplifier, this will become $-10\log_{10}(\alpha)\text{db}$ (positive) or $\frac{1}{\sqrt{\alpha}}$ in abs).

Characteristics

Lead network has high pass filter characteristics. Compensator zero is closer to the origin than pole. The reshaping of root locus/frequency response is achieved through the phase lead characteristics. It improves damping ratio and hence speed of response, ie, rise time and settling time reduces. Gain cross over frequency increase increasing bandwidth and speed of closed loop response. It also increases phase margin. Additional amplifier gain is essential to overcome low frequency attenuation. There is no much improvement in steady state performance.

Main drawback of lead compensator is if the system is prone to the effect of noise, which is of high frequency signal, then this may amplify the noise and performance may degrade.

Problem 1:

For a unity feedback control system with $G(s) = \frac{4K}{s(s+2)}$, design a phase lead compensator such that the closed loop system will satisfy the following requirements. $K_v=20/s$, Phase margin= 50° , gain margin $>10\text{db}$.

Procedure: Design a Phase *lead* compensator for the given OLTF

Specification: Phase margin at least 50 degrees, Velocity error coefficient is 20/s

1. To evaluate K of given OLTF:

Given $K_v=20$.

Where $K_v = \lim_{s \rightarrow 0} sGH(s)$, which gives K as G(s) is type 1 system.

Draw Bode plot using Matlab.

%enter the transfer function of uncompensated system

numg=[4];

deng=[1 2 0];

G=tf(numg,deng);

%To solve for K satisfying $K_v=20$

K=20/(dcgain(conv([1 0],numg),deng));

1. With this value of K, obtain Bode plot of OLTF, from which determine Phase margin
 pm
 %to obtain the factored form of G *K
 G1=zpk(G*K);
 w=logspace(-1,2,100);
 bode(G1,w);
 grid on;
 Design of Lead network:

To compute the max phase lead angle considering 8 to 12deg additional for tolerance

Measure actual phase margin as pm from graph
 Compute desired max phase lead ϕ_m as PC=(PM+10)-pm;

Design of lead compensator by computing alpha
 $\alpha = (1 - \sin(PC \cdot \pi / 180)) / (1 + \sin(PC \cdot \pi / 180))$; %($\alpha < 1$)

and corresponding magnitude in dB contributed by lead network at max phase lead angle frequency
 $\text{magpc} = 20 \cdot \log_{10}(1/\sqrt{\alpha})$; (Positive)(after using amplifier)

To obtain the frequency corresponds to the magnitude magpc fom graph
 From the Bode plot of gain compensated system measure frequency at which gain is -magpc dB(negative)

Note this frequency as wmax. Compute zero of lead network as

2. Determine time constant of compensating network $\tau = \frac{1}{(\sqrt{\alpha})\omega_m}$ or

zc=wmax*sqrt(alpha);

3. Pole of lead network as pc=zc/alpha;

4. Obtain the TF of compensating network $G_c(s) = \frac{1}{\alpha} \frac{\left(s + \frac{1}{\tau}\right)}{\left(s + \frac{1}{\alpha\tau}\right)}$

Verification using MATLAB:

kc=1/alpha;

GC=tf(kc*[1 zc],[1 pc]);%Lead network transfer function with amplifier included

%compensated system

CGC=G1*GC;%Overall forward path transfer function

margin(CGC);%measure compensated system Phase margin, gain margin

pause

S=tf([1 0],[0 1]);

KV=dcgain(S*CGC)%Verification of velocity error coefficient

%CLOSED LOOP SYSTEM

CLGC=feedback(CGC,1);

step(CLGC) %closed loop step response of compensated unity feedback system

%factorise the compensated system

CGC1=zpk(CGC)

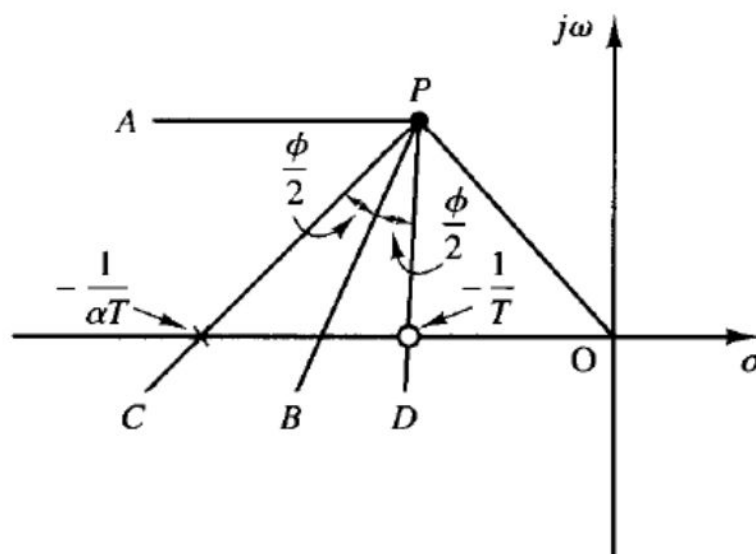
LEAD Compensator design using Root locus

Procedure

From the performance specifications, determine the desired location for the dominant closed loop poles. After drawing root locus ascertain whether by gain adjustment alone yields the desired roots. If not find the sum of angles at one of the desired closed loop poles with the open loop poles and zeros of the original system, and determine the angle of deficiency ϕ_m to be added so that the total sum of the angles is equal to $\pm 180^\circ + (2k+1)180^\circ$. The lead compensator must contribute this angle ϕ_m to make the desired closed loop poles pass through the root locus. The lead compensator $G_c(s)$ has the form

$$G_c(s) = K_c \alpha \frac{Ts + 1}{\alpha Ts + 1} = K_c \frac{s + \frac{1}{T}}{s + \frac{1}{\alpha T}}, \quad (0 < \alpha < 1)$$

, where K_c is the amplifier gain required to compensate low frequency attenuation, α and T are determined from angle deficiency. K_c is determined from the requirement of open loop gain.



First draw a horizontal line passing through point P, the desired location for one of the dominant closed loop poles. This is shown as line PA. Draw also a line connecting point P and the origin. Bisect the angle between the lines PA and PO. Draw two lines PC and PD that make angles $\pm \phi_m/2$ with the bisector PB. The intersections of PC and PD with the negative real axis give necessary location for the pole and zero of the lead network. The compensator thus designed will make point P a point on the root locus of the compensated system.

If static error coefficients are not specified, the lead compensator pole and zero thus obtained will contribute the necessary angle ϕ_m . If no other constraints are imposed on the system,

make α as large as possible to minimize steady state error. The open loop gain is determined by use of the magnitude condition.

Once the lead compensator is designed, check whether the specifications are met with cascading compensator with the original system. If not satisfactory, repeat the design by changing the position of compensator pole and zero till performance is satisfactory.

Exercise 1:

%OGATA pg 486-490 ex 10-1 correct solution

%problem OGATA $4/s(s+2)$; spec desired clpole $\omega_n=4, z=0.5$

```
clear all;close all;clc
%desired zeta and natural frequency
z=0.5; wn=4;
n1=4; d1=conv([1 0],[1 2])
G=tf(n1,d1);
figure(1)
rlocus(G)
%desired closed loop poles
s1=-z*wn+j*wn*sqrt(1-z*z); s2=-z*wn-j*wn*sqrt(1-z*z);
%to determine angle contribution at desired cl poles
th1=(angle(polyval([1 0],s1)))*180/pi
th2=(angle(polyval([1 2],s1)))*180/pi
phd=180-(th1+th2)
%phase angle lead required by lead network is -phd
%graphical construction
%angle between line joining s1 with origin and horizontal from s1
phs1=(angle(polyval([1 0],s1)))*180/pi
%zero location bisection of above angle and half of lead contribution cw
angz=(phs1+phd)/2;
%pole location bisection of phs1 and half of lead angle ccw
angp=(phs1-phd)/2;
%to find x coordinate of zero and pole with respect to vertical from s1
zang=(angz-(phs1-90))*pi/180;%angle of zero from vertical from s1
pang=(angp-(phs1-90))*pi/180;%angle of pole from vertical from s1
%zero and pole location with respect to real part of s1
x1=imag(s1)*tan(zang)%zero location with respect to vertical from s1
x2=imag(s1)*tan(pang)%pole position wrt vertical from s1
ze=-(real(s1)-x1)
pe=-(real(s1)-x2)
%LEAD COMPENSATOR
nc=[1 ze]; dc=[1 pe];
gc=tf(nc,dc);%
cgc=G*gc;
zpk(cgc)
% to find gain
[k, p]=rlocfind(cgc,s1)
```

```

cgcs=k*cgc
figure(2)
rlocus(cgc)
T=feedback(cgc*k,1)
figure(3)
step(T)
d=tf([1 0],[0 1])
kv=dcgain(d*cgcs)
%comparison of equivalent second order system with dominant poles s1,s2
dcl=[1 -(s1+s2) s1*s2]; ncl=s1*s2;
hold on
tfcl=tf(ncl,dcl)
step(tfcl)

```

Exercise 2:

% When rise time, overshoot and ω_n is specified

```

%problem pillai 100/s(s+8); spec OS 9.5%  $\omega_n$  12r/s
clear all;close all;clc
n1=[100]; d1=[1 8 0];
G=tf(n1,d1);
figure(1)
rlocus(G)
OS=9.5; $\omega_n$ =12;
%desired zeta
z=(-log(OS/100))/(sqrt(pi*pi+log(OS/100)^2));
%desired closed loop poles
s1=-z* $\omega_n$ +j* $\omega_n$ *sqrt(1-z*z);
s2=-z* $\omega_n$ -j* $\omega_n$ *sqrt(1-z*z);
%to determine angle contribution at desired cl poles
th1=(angle(polyval([1 0],s1)))*180/pi
th2=(angle(polyval([1 8],s1)))*180/pi
phd=180-(th1+th2)
%phase angle lead required by lead network is -phd
%graphical construction
%angle between line joining s1 with origin and horizontal from s1
phs1=(angle(polyval([1 0],s1)))*180/pi
%zero location bisection of above angle and half of lead contribution cw
angz=(phs1+phd)/2;
%pole location bisection of phs1 and half of lead angle ccw
angp=(phs1-phd)/2;
%to find x coordinate of zero and pole with respect to vertical from s1
zang=(angz-(phs1-90))*pi/180;%angle of zero from vertical from s1
pang=(angp-(phs1-90))*pi/180;%angle of pole from vertical from s1
%zero and pole location with respect to real part of s1
x1=imag(s1)*tan(zang)%zero location with respect to vertical from s1
x2=imag(s1)*tan(pang)%pole position wrt vertical from s1
ze=-(real(s1)-x1)
pe=-(real(s1)-x2)
%LEAD COMPENSATOR

```

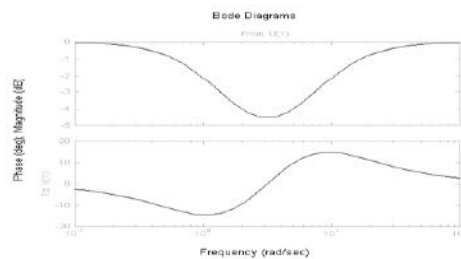
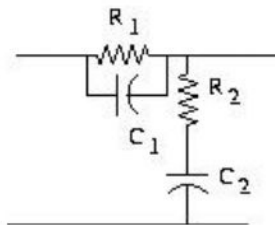
```

nc=[1 ze];
dc=[1 pe];
gc=tf(nc,dc);%
cgc=G*gc;
zpk(cgc)
% to find gain
[k, p]=rlocfind(cgc,s1)
cgcs=k*cgc
figure(2)
rlocus(cgc)
T=feedback(cgc*k,1)
figure(3)
step(T)
d=tf([1 0],[0 1])
kv=dcgain(d*cgcs)
%comparison of equivalent second order system with dominant poles s1,s2
dcl=[1 -(s1+s2) s1*s2]; ncl=s1*s2;
hold on
tfcl=tf(ncl,dcl)
step(tfcl)

```

L3c: LAG-LEAD compensator design

Lag-Lead network



$$TF = \frac{\left(s + \frac{1}{\tau_1}\right) \left(s + \frac{1}{\tau_2}\right)}{\left(s + \frac{1}{\beta\tau_1}\right) \left(s + \frac{1}{\alpha\tau_2}\right)} \quad \alpha\beta = 1, \alpha < 1, \beta > 1, R_1C_1 = \tau_1, R_2C_2 = \tau_2$$

Lag Lead

Lag-lead compensator: Used to achieve performance of both Lag compensator and lead compensator in overall system.

Problem 1:

For a unity feedback control system with $G(s) = \frac{K}{s(s+8)(s+30)}$, design a lag - lead compensator such that the closed loop system will satisfy the following requirements. $K_v=20/s$, peak time=0.6sec and% overshoot 15.

%Design of Lag -LEAD compensator using Frequency domain(bode plot) method.

%Enter the uncompensated system transfer function

numg=[0 0 0 1];deng=conv(conv([1 0],[1 8]),[1 30]);

G=tf(numg,deng);

%enter the design specifications(overshoot, peak time and KV)

TP=0.6;KV=20;OS=15;

%TO FIND k,use sG(s)H(s)

K=KV/(dcgain(conv([1 0],numg),deng));

%To find the gain margin, phase margin, gain cross over frequency

%and phase cross over frequency

%To find the desired bandwidth

z=(-log(OS/100))/(sqrt(pi^2+log(OS/100)^2));

wn=pi/(sqrt(1-z^2));

wb=wn*sqrt((1-2*z^2)+sqrt(4*z^4-4*z^2+2));

wb1=wb*0.8;

pmreq=atan(2*z/(sqrt(-2*z^2+sqrt(1+4*z^4))))*(180/pi);

w=logspace(-1,2,100);

[m,P]=bode(K*G,wb*0.8);

pmreqc=pmreq-180+P;

beta=(1-sin(pmreqc*pi/180))/(1+sin(pmreqc*pi/180));

%design og lag compensator

%adjust for design ,first take 10 and then adjust for design

W1=wb1/8;

W2=W1*beta;

kclag=beta;

%lag compensator is

LAGC=tf(kclag*[1 W1],[1 W2]);

%margin(LAGC)

%TO design the lead portion of the compensator

%Lead compensator transfer function

P1=wb1*sqrt(beta);

P2=P1/beta;

kclead=1/beta;

LEADC=tf(kclead*[1 P1],[1 P2]);

%margin(LEADC)

%TO obtain the transfer function of compensated system

C=K*G*LEADC*LAGC;

pause

margin(C)

%TO check KV,

D=tf([1 0],[0 1]);

C1=C*D; KV=dcgain(C1)

pause

%to obtain the closed loop step response

```
C2=feedback(C,1);  
step(C2)
```

Exercise 1:

1. For a unity feedback control system with $G(s) = \frac{K}{s^3 + 5s^2 + 4s}$, design a lag - lead compensator such that the closed loop system will satisfy the following requirements.
 $K_v=10/s$, phase margin= 50° , gain margin $>10\text{db}$.

L4a: Signal Generation and Sampling Rate Conversion

Aim: To generate and plot various types of signals.

Problem 1:

Generate the following function or signals: delta, step input, exponential signal and sinusoidal signal.

% 1. Delta Function

```
clc;
close all;
clear all;

n=0:10; n0=2;
x=[(n-n0)==0];
subplot(2,2,1);
stem(n,x);
grid on;
title('delayed impulse');
```

% 2. Step Function

```
n=0:10; n0=4;
x=[(n-n0)>=0];
subplot(2,2,2);
stem(n,x);
grid on;
title('delayed step');
```

% 3. Exponential Function

```
subplot(2,2,3);
n=0:1:15;
x=(0.8).^n;
stem(n,x); grid on;
title('0.8^n');
```

% 4. Sinusoidal sequence

```
subplot(2,2,4);
n=0:0.1:5;
x = 3*sin(pi*n-pi/4); %+ 2*sin(0.5*pi*n);
stem(n,x);
grid on;
title('Sinusoidal sequence');
```

Problem 2:

Using $D = 2$ and $x(n) = \{1, 2, 3, 4, 3, 2, 1\}$ write a MATLAB program to verify that the downsampler is time varying.

```
clc;
clear all;

x = [1,2,3,4,3,2,1];
y = downsample(x,2)

x = [1,2,3,4,3,2,1];
y = downsample(x,2,1)
```

Exercise 1:

Generate and plot

$x(n) = \{\dots, 5, 4, 3, 2, 1, 5, 4, 3, 2, 1, 5, 4, 3, 2, 1, \dots\}$; $-10 \leq n \leq 9$.

**Exercise 2:**

Generate $x(n) = e^{(-0.1 + j0.3)n}$, $-10 \leq n \leq 10$, plots its real part, and the imaginary part.

Exercise 3:

Let $x(n) = u(n) - u(n - 5)$. Plot its even and odd components.

Exercise 4:

Let $I = 2$ and $x(n) = \{1, 2, 3, 4\}$. Write a MATLAB program to verify that the upsampler is time varying.

```
clc;
clear all;
x = [1,2,3,4];

v = upsample(x,3);

v = upsample(x,3,1);

v = upsample(x,3,2)
```

L4b: Convolution of Signals

Aim: Aim: To find the linear and circular convolution of given sequences.

Problem 1:

Write a MATLAB program to perform convolution of the following two discrete time signals.

$x_1(i) = 1; 0 < i < 5$ and $x_2(i) = 1$ for $1 < i < 5$

```
clc;
clear all;
close all;

n=0:5;
x1=1.*(n>=1 & n<=4); %generate signal x1(n)
x2=1.*(n>=2 & n<=4); %generate signal x2(n)
N1=length(x1);
N2=length(x2);
x3=conv(x1,x2); %perform convolution of signals x1(n) and x2(n)
n1=0:1:N1+N2-2;
%To plot the results
subplot(3,1,1); stem(n,x1);
xlabel('n');ylabel('x1(n)'); title('signal x1(n)');
subplot(3,1,2); stem(n,x2);
xlabel('n');ylabel('x2(n)'); title('signal x2(n)');
subplot(3,1,3); stem(n1,x3);
xlabel('n');ylabel('x3(n)');
title('signal x1(n)*x2(n)');
```

Problem 2:

Write a MATLAB program to perform circular convolution of the following two discrete time signals.

$x_1(n) = [1 \ 2]$ and $x_2(n) = [1 \ 2 \ 3 \ 4]$

```
clc;
clear all;
close all;
x1=input('Enter the first sequence x1[n]= '); x2=input('Enter the second sequence x2[n]='); n
= max(length(x1),length(x2));
Y=cconv(x1,x2,n); %perform circular convolution
stem(Y);
xlabel('n'),ylabel('Y');
title('Plot of circularly convoluted sequence');
```

Exercise 1:

Write a MATLAB program to perform convolution of the following two discrete time signals.

$$x_1(n) = [1 \ 2 \ -1 \ 1] \text{ and } x_2(n) = [1 \ 0 \ 1 \ 1]$$

Exercise 2:

Write a MATLAB program to verify associative property of convolution. Take any two sequences $x_1(n)$ and $x_2(n)$. Verify that $x_1(n)*x_2(n) = x_2(n)*x_1(n)$.

L4c: Z transform**Problem 1:**

Write a MATLAB program to find Z-transform of the following causal signals.

a) 0.5^n , b) n , c) na^n .

```
clear all;
syms n a real; %Let n be real variable
% a)
x1=0.5^n;
disp('z-transform of "0.5^n" is') ; X1 =ztrans(x1)
% b)
x2=n;
disp('z-transform of "n" is') ; X2 =ztrans(x2)
% c)
x3=n*(a^n);
disp('z-transform of "n*(a^n)" is') ; X3 =ztrans(x3)
```

Problem 2:

Given a causal system , $y(n) = 0.9y(n - 1) + x(n)$. Determine the following using MATLAB

- i) $H(z)$ and sketch its pole-zero plot.
- ii) The impulse response $h(n)$.

```
clear all;
close all;
clc;

syms n real;
syms z complex; b = [1, 0];
```

```

a = [1, -0.9];
zplane(b,a);
H=1/(1-0.9*(z^(-1)));

disp('Impulse response h(n) is');
h=iztrans(H) %compute impulse response

i=0;
for i=1:1:21
    n=(i-1);
    n1(i) = n;
    hn = eval(h);
    hn1(i) = hn;

end
figure(2);
stem(n1,hn1,'b+');
grid on;
xlabel('n');
ylabel('h(n)');

```

Exercise 1:

Let $X_1(Z) = 1+2Z^{-1}+3Z^{-2}$ and $X_2(Z) = 1+3Z^{-1}+4Z^{-2}$. Determine $X_3(Z) = X_1(Z) X_2(Z)$ using MATLAB.

Exercise 2:

Write a MATLAB program to find inverse Z-transform of the following Z-domain signal.

$$X(z) = \frac{1}{1 - 1.5z^{-1} + 0.5z^{-2}}$$

L5a: DFT

Problem 1:

Let $x(n) = (0.7)^n u(n)$. Sample its z-transform on the unit circle with $N = 5$ and study its effect in the time domain.

$$X(z) = \frac{1}{1 - 0.7z^{-1}} = \frac{z}{z - 0.7}, \quad |z| > 0.7$$

We can now use MATLAB to implement the sampling operation

$$\hat{X}(k) = X(z)|_{z=e^{j2\pi k/N}}, \quad k = 0, \pm 1, \pm 2, \dots$$

and the inverse DFS computation to determine the corresponding time-domain.

Create a function file `idfs.m` and write the below code:

```
function [xn] = idfs(Xk,N)
% Computes Inverse Discrete Fourier Series n = [0:1:N-1]; % row vector for n
k = [0:1:N-1]; % row vector for k WN = exp(-j*2*pi/N); % Wn factor
nk = n*k; % creates a N by N matrix of nk values WNnk = WN.^(-nk); % IDFS
matrix
xn = (Xk * WNnk)/N; % row vector for IDFS values
```

Create a new script and call the function.

```
N = 5;
k = 0:1:N-1; % sample index wk = 2*pi*k/N;
zk = exp(j*wk); % samples of z
Xk = (zk)/(zk-0.7); % DFS as samples of X(z) xn = real(idfs(Xk,N)); % IDFS
xtilde = xn*ones(1,8); xtilde = (xtilde(:))'; % Periodic sequence subplot(2,2,1);
stem(0:39,xtilde);axis([0,40,-0.1,1.5])
xlabel('n'); ylabel('xtilde(n)'); title('N=5')
```

Problem 2:

Write a MATLAB program to perform circular convolution of the discrete time sequences $x1(n) = \{0, 1, 0, 1\}$ and $x2(n) = \{1, 2, 1, 2\}$ using DFT.

% Program to perform Circular Convolution via DFT

`clc`

`clear all;`

`N = 4; % declare the value of N`

`x1 = [0,1,0,1]; % declare the input sequences x2 = [1,2,1,2];`

`disp('The 4-point DFT of x1(n) is');`

`X1 = fft(x1,N) % compute 4-point DFT of x1(n) disp('The 4-point DFT of x2(n) is,');`

`X2 = fft(x2,N) % compute 4-point DFT of x2(n) disp('The product of DFTs is,');`

```
X1X2 = X1.*X2      % product of DFTs disp('Circular convolution of x1(n) and x2(n) is,');
X3 = ifft(X1X2)    % perform IDFT to get result of circular convolution
```

Exercise 1:

Let $x(n) = 10(0.8)^n$, $0 \leq n \leq 10$. Determine and plot $x((-n))_{11}$.

```
n = 0:10;
x = 10*(0.8).^n;
y = x(mod(-n,11)+1);

subplot(2,1,1); stem(n,x);
title('Original sequence')
xlabel('n');
ylabel('x(n)');

subplot(2,1,2); stem(n,y);
title('Circularly folded sequence');
xlabel('n');
ylabel('x(-n mod 10)');
```

L5b: FIR Filter Design

Problem 1:

Let $h(n) = \{-4, 1, -1, -2, 5, 6, 5, -2, -1, 1, -4\}$. Write a MATLAB program to plot amplitude response $H_r(\omega)$ and the locations of the zeros of $H(z)$.

Function:

```
function [Hr,w,a,L] = Hr_Type1(h)

M = length(h); L = (M-1)/2;
a = [h(L+1) 2*h(L:-1:1)]; % 1x(L+1) row vector
n = [0:1:L]; % (L+1)x1 column vector
w = [0:1:500]'*pi/500; Hr = cos(w*n)*a';
```

Main code:

```

h = [-4,1,-1,-2,5,6,5,-2,-1,1,-4];
M = length(h); n = 0:M-1;
[Hr,w,a,L] = Hr_Type1(h);
a,L
%a = 6 10 -4 -2 2 -8
%L = 5
amax = max(a)+1; amin = min(a)-1;
subplot(2,2,1); stem(n,h); axis([-1 2*L+1 amin amax])
xlabel('n'); ylabel('h(n)'); title('Impulse Response')
subplot(2,2,3); stem(0:L,a); axis([-1 2*L+1 amin amax])
xlabel('n'); ylabel('a(n)'); title('a(n) coefficients')
subplot(2,2,2); plot(w/pi,Hr);grid
xlabel('frequency in pi units'); ylabel('Hr')
title('Type-1 Amplitude Response')
h=double(h);
subplot(2,2,4); zplane(h,1);

```

Exercise 1:

Let $h(n) = \{-4, 1, -1, -2, 5, 6, 6, 5, -2, -1, 1, -4\}$. Write a MATLAB program to plot amplitude response $H_r(\omega)$ and the locations of the zeros of $H(z)$.

Exercise 2:

Write a MATLAB program to determine the impulse response of FIR lowpass Filter using rectangular window (taking 7 samples of window sequence and with a cutoff frequency, $w_c = 0.2\pi$ rad/sample) and hence plot the frequency response.

```

wc=0.2*pi
N = 7 ;
hd = zeros (1, N);
a=(N-1)/2;
hna=wc/pi;

%% rect window
k = 1:1:((N-1)/2);
n = k-1-((N-1)/2) ;
hd( k ) = (sin(wc*n))./(pi*n) ;
hn(k) = hd(k) ;
hn = [hn hna];

%%
a = ( N - 1 ) / 2 ;
w= 0 :pi/16 : pi;
Hw1 = hna * exp (-j*w*a) ;
Hw2 = 0 ;
for m=1:1:a
Hw3= hn(m)*((exp(j*w*(1-m)))+(exp(-j*w*(1-m+2*a))));
Hw2=Hw2+Hw3;
end
Hw = Hw2 + Hw1
H_mag = abs( Hw )
plot ( w/pi,H_mag, 'k' ); grid ;
title ( 'Magnitude Response', 'fontweight' , 'b' ) ;
xlabel ( ' Normalized Freq., \omega/\pi', 'fontweight' , 'b' ) ;
ylabel ( 'Magnitude' , 'fontweight' , 'b' ) ;

```

L5c: IIR Filter Design

Problem 1:

Design Butterworth filter using Bilinear transformation and Impulse invariant transformation for

i) Sampling frequency of 8000 Hz, ii) normalized passband frequency of 0.25 rad/sample, iii) normalized stop band frequency of 0.375 rad/sample, iv) pass band ripple of 0.5 dB, and v) stop band ripple of 30 dB.

```

clear; clc; close all;
Fs=8000;
wp=1000*2/Fs; ws=1500*2/Fs;
wpa=1000*2*pi; wsa=1500*2*pi;
Rp=0.5; Rs=30;
% Butterworth filter

```

```

% 1. Bilinear transformation
[Nb1,wn_b1]=buttord(wp,ws,Rp,Rs);
[num_but1,den_but1]=butter(Nb1,wn_b1);
% 2. Impulse invariant transformation
[Nb2,wn_b2]=buttord(wpa,wsa,Rp,Rs,'s');
[num_but2a,den_but2a]=butter(Nb2,wn_b2,'s');
[num_but2,den_but2]=impinvar(num_but2a,den_but2a,Fs);
f=0:10:Fs/2;
hb1=freqz(num_but1,den_but1,f,Fs);
hb2=freqz(num_but2,den_but2,f,Fs);
plot(f,20*log10(abs(hb1)),f,20*log10(abs(hb2)));
Title('Butterworth filter');axis([0 4000 -60 1]);
figure; plot(f,abs(hb1),f,abs(hb2));
Title('Butterworth filter');

```

Exercise 1:

Design Chebyshev type I and Chebyshev type II filters using Bilinear transformation and Impulse invariant transformation for

i) Sampling frequency of 8000 Hz, ii) normalized passband frequency of 0.25 rad/sample, iii) normalized stop band frequency of 0.375 rad/sample, iv) pass band ripple of 0.5 dB, and v) stop band ripple of 30 dB.

```

% Chebyshev I filter
% 1. Bilinear transformation
[Nc11,wn_c11]=cheblord(wp,ws,Rp,Rs);
[num_c11,den_c11]=cheby1(Nc11,Rp,wn_c11);
% 2. Impulse invariant transformation
[Nc12,wn_c12]=cheblord(wpa,wsa,Rp,Rs,'s');
[num_c12a,den_c12a]=cheby1(Nc12,Rp,wn_c12,'s');
[num_c12,den_c12]=impinvar(num_c12a,den_c12a,Fs);
f=0:10:Fs/2;
hc11=freqz(num_c11,den_c11,f,Fs);
hc12=freqz(num_c12,den_c12,f,Fs);
figure;plot(f,20*log10(abs(hc11)),f,20*log10(abs(hc12)));
Title('Chebyshev I filter');axis([0 4000 -60 1]);
figure; plot(f,abs(hc11),f,abs(hc12));
Title('Chebyshev I filter');

```

```

% Chebyshev II filter
% 1. Bilinear transformation
[Nc21,wn_c21]=cheb2ord(wp,ws,Rp,Rs);
[num_c21,den_c21]=cheby2(Nc21,Rs,wn_c21);
% 2. Impulse invariant transformation
[Nc22,wn_c22]=cheb2ord(wpa,wsa,Rp,Rs,'s');

```

```

[num_c22a,den_c22a]=cheby2(Nc22,Rs,wn_c22,'s');
[num_c22,den_c22]=impinvar(num_c22a,den_c22a,Fs);
f=0:10:Fs/2;
hc21=freqz(num_c21,den_c21,f,Fs);
hc22=freqz(num_c22,den_c22,f,Fs);
figure;plot(f,20*log10(abs(hc21)),f,20*log10(abs(hc22)));
Title('Chebyshev II filter');axis([0 4000 -60 1]);
figure; plot(f,abs(hc21),f,abs(hc22));
Title('Chebyshev II filter');

```

L6a: Speed control of DC motor using FOPDT model

Aim: This experiment will model the DC motor by simply applying a step input to the system and observing its time domain response. Based on the transient response, the DC motor will be approximated as a first order transfer function with time delay, and its parameters will be determined consequently

Theory

In empirical model building, models are determined by making small changes in the input variable(s) about a nominal operating condition. The resulting dynamic response is used to determine the model. This general procedure is essentially an experimental linearization of the process that is valid for some region about the nominal conditions. In this experiment, the parameters of this transfer function are experimentally obtained using step response tests.

The method involves the following four actions:

1. Allow the process to reach steady state.
2. Introduce a single step change in the input variable.
3. Collect input and output response data until the process again reaches steady state.
4. Perform the calculations.

The form of the model is as follows, with $U(s)$ denoting the input and $Y(s)$ denoting the output, both expressed in deviation variables:

$$G(s) = \frac{\Delta Y(s)}{\Delta U(s)} = \frac{Ke^{-t_d s}}{\tau s + 1} \quad (1)$$

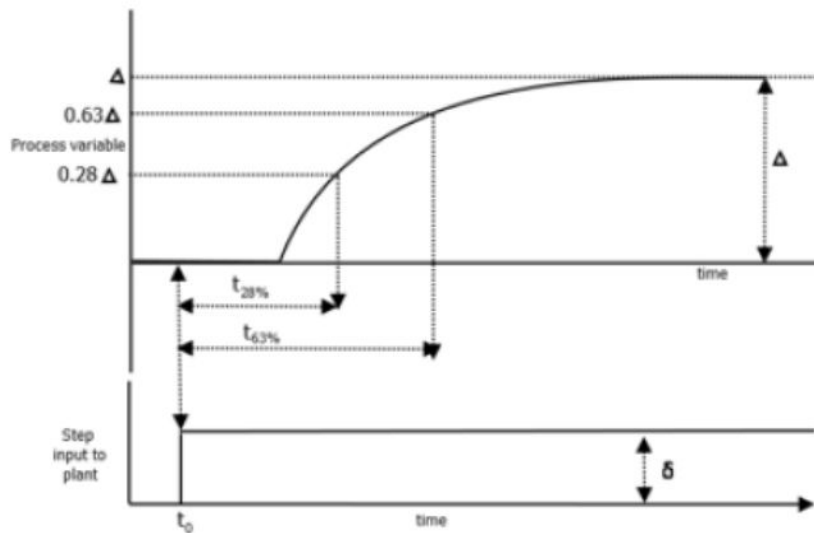


Fig: Step response

The intermediate values as can be seen from the graph are the magnitude of the input change, δ ; the magnitude of the steady-state change in the output, Δ ; and the times at which the output reaches 28.3 and 63.2 percent of its final value. The values from the plot can be related to the model parameters using the general expression in equation (1). Any two values of time can be selected to determine the unknown parameters, τ and t_d . The typical times are selected where the transient response is changing rapidly so that the model parameters can be accurately determined in spite of measurement noise

The parameters of the transfer function are calculated using the two-point method as follows:

$$K = \frac{\text{Difference in two steady states of output}}{\text{Difference in two steady states of input}}$$

$$\tau = 1.5 (t_{63.2} - t_{28.3})$$

$$t_d = t_{63.2} - \tau$$

By this we have an approximate model for the motor.

Procedure

1. Go to the experiment application file (Automated_step_test), right click on it and run the file as an administrator.
2. Then, click on Play button, on the top left corner of the interface
3. If the file is running the Play button will be as shown below;
4. Select the appropriate COM port to which the USB-serial cable is connected;
5. Next, click on the RUN button;
6. In the 'open-loop' mode, input $u(t)$ to the motor is in terms of PWM and the output $y(t)$ is the speed in revolutions per minute (RPM). The motor is brought to an operating point by applying a suitable input and the plant output is allowed to settle at this value.
7. From this position, a step of appropriate magnitude is applied to the plant and the motor speed is allowed to settle again.
8. The experiment will run for a time of 5 seconds, and then the motor will automatically stop, and you will have the respective graphs of input and output.
9. Once the motor stops, click on STOP button;
10. After the stop button is clicked, a dialogue box showing the file path will be displayed on the interface; (Do not click the RETURN button)
11. This is the file in which the user will have the corresponding step test. Navigate to the location to access the file. It will have three columns, the first one being the time instants, second column is the input in terms of PWM and third column consists of the output speed of the dc motor in RPM.
12. The First-order Plus Time Delay (FOPTD) model is given by

$$G(s) = \frac{\Delta Y(s)}{\Delta U(s)} = \frac{Ke^{-t_d s}}{\tau s + 1}$$

We have to find parameters: gain K , time constant τ , and dead time t_d by carrying out a step-test on the DC motor.

13. Apply two-point method for system identification. From the step test data, find out

$t_{63.2}$ = Time required for the output to reach 63.2 % of the steady-state value

$t_{28.3}$ = Time required for the output to reach 28.3 % of the steady-state value.

Care has to be taken to measure the time from the instant of application of step Input.

14. The parameters of the transfer function (6.1) are calculated using the two-point method as follows:

$$K = \frac{\text{Difference in two steady states of output}}{\text{Difference in two steady states of input}}$$

$$\tau = 1.5 (t_{63.2} - t_{28.3})$$

$$t_d = t_{63.2} - \tau$$

15. The obtained model is validated against the plant response by applying the same step input to the model and comparing the responses of the plant and the model.

Sample Results

1. The First-order Plus Time Delay (FOPTD) model is given by

$$\frac{\Delta Y(s)}{\Delta U(s)} = \frac{K e^{-t_d s}}{\tau s + 1}$$

We have to find parameters K , τ and t_d by carrying out a step-test on the DC motor.

2. In the open loop, the plant is brought to equilibrium by applying a step of 150 PWM units. The corresponding speed is around 8000 RPM, see Fig. 2.

3. After the motor speed settles, the PWM input is instantaneously changed to 170. As a result, the speed increases to around 9000 RPM and settles there, as shown in Fig. 2.

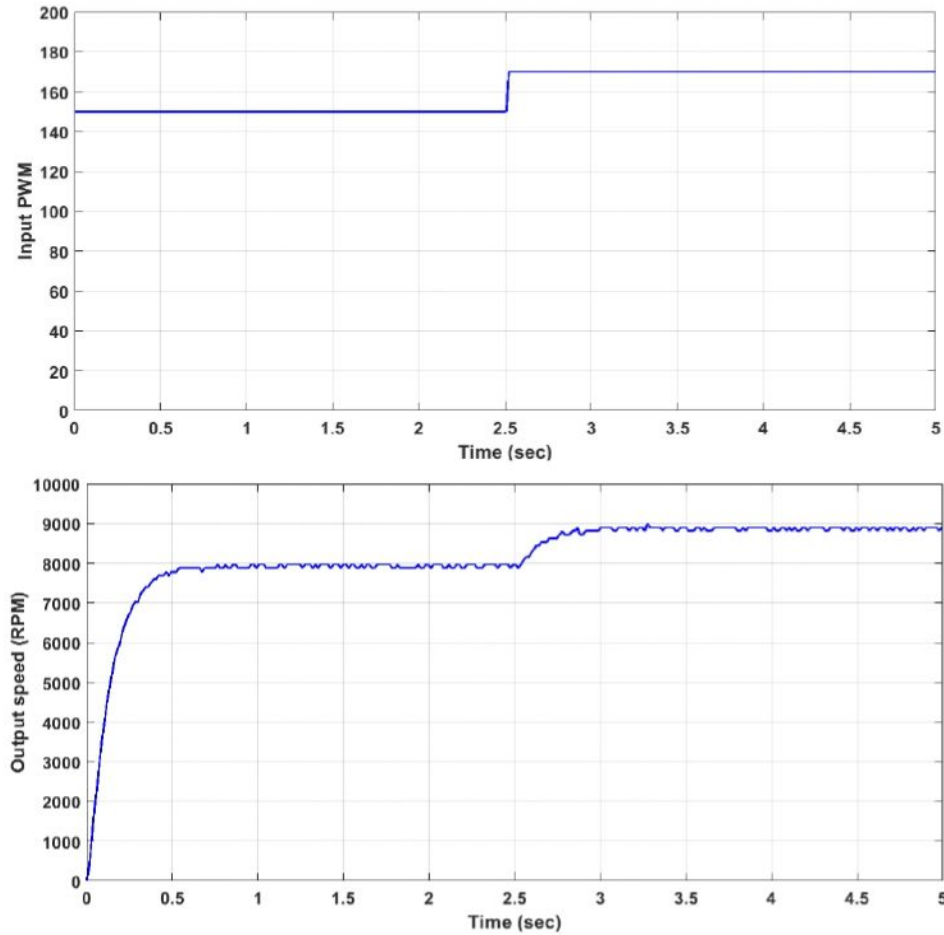


Fig: Step-test data

4. Following are noted

$$t_{63.2} = 0.135 \text{ sec} \quad t_{28.3} = 0.062 \text{ sec} \quad \Delta u(t) = 20 \text{ PWM units} \quad \Delta y(t) = 937.5 \text{ RPM}$$

5. Using the two-point method, we get the parameters of the transfer function (1.1) as

$$K = 51.5625 \quad \tau = 0.1095 \text{ sec} \quad L = 0.0255 \text{ sec}$$

6. The model is

$$\frac{\Delta Y(s)}{\Delta U(s)} = \frac{51.5625 \exp^{-0.0255s}}{0.1095s + 1}$$

7. The obtained model is validated against the plant response by applying the same step input to the model and comparing the responses of the plant and the model. It should be noted that the transfer function (1) is a linearized perturbation model around the operating point of 150

PWM, 8000 RPM. Hence for model validation, we use the deviation values of the input and output signals

$$\Delta u(t) = u(t) - 150$$

$$\Delta y(t) = y(t) - 8000$$

PI Control Gains for Motor Speed Control

Aim: The purpose of this experiment is to design and implement a Proportional-Integral (PI) controller for the speed control of DC motor.

Theory

The PID controller is by far the most common control algorithm. Most industrial feedback loops are based on PID control or some minor variations of it. The output of a PID controller which is the control input to the plant, in the time-domain is as follows:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (2)$$

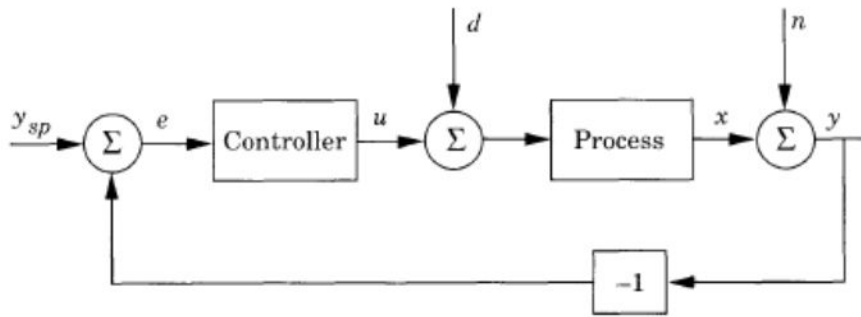


Fig: DC Motor PI speed control closed-loop block diagram

where u is the control signal and e represents the tracking error i.e. the difference between the desired input value and the actual output. This error signal e will be sent to the PID controller. The control signal u to the plant is equal to the proportional gain (K_p) times the magnitude of the error plus the integral gain (K_i) times the integral of the error plus the derivative gain (K_d) times the derivative of the error. This control signal u is sent to the plant, and the new output is obtained. The new output is then fed back and compared to the reference to find the new error signal e . The controller takes this new error signal and computes its derivative and its integral again, ad infinitum.

The controller can also be parameterized as

$$u(t) = K_p \left(e(t) + \frac{1}{T_i} \int_0^t e(\tau) d\tau + T_d \frac{de(t)}{dt} \right) \quad (3)$$

Where T_i is the integral time constant and T_d is the derivative time constant.

The speed of the DC motor is controlled using a Proportional-Integral control system.

Procedure

1. First, go to the experiment file, right click on it and run the file as an administrator.
2. Then, click on Play button, on the top left corner of the interface
3. If the file is running the Play button will be as shown below;
4. Select the appropriate COM port to which the USB-serial cable is connected;

5. Next, click on the RUN button;
6. For sample PI values, click on the Sample PID values button;
7. Make sure that the motor is brought to the same operating point for which the FOPDT model was identified and is settled there. Since we are now working in the closed –loop configuration, the operating point is reached by giving a set point in RPM.
8. Give a new appropriate set point in RPM and observe that motor speed tracks the new set point and settles there. The set point can be varied from the Set Point control;
9. To check the controller performance, a negative step in the desired speed may also be given to see if the motor speed decreases accordingly and reaches to the new set point.
10. Once you are done with the experiment, click on STOP button;

Sample Results

1. We use the AMIGO tuning rule to tune the controller given in the book: Handbook of PI and PID Controller Tuning Rules by A. O'Dwyer (Imperial College Press UK,2009). The PI parameters are given by

$$K_c = ((0.15/K) + (0.35 - ((L * \tau)/(L + \tau)^2)) * (\tau / (K * L)))$$

$$T_i = ((0.35 * L) + (6.7 * L * \tau^2) / (\tau^2 + 2 * \tau * L + 10 * L^2))$$

2. Using these relations, we get $K_c = 0.0193$ and $T_i = 0.0016$ minutes. These values are used for the implementation of PI controller.

3. After the speed settles at 8000 RPM, a step of 1000 RPM is applied. So the set point is 9000 RPM. The controller is implemented and the results are shown in Fig.6.5. It is seen that the output follows the set point and the speed settles at 9000 RPM.

4. Next, a negative step of 1000 RPM is applied. It is clearly observed from Fig. 6.4 that motor speed decreases and settles at 8000 RPM.

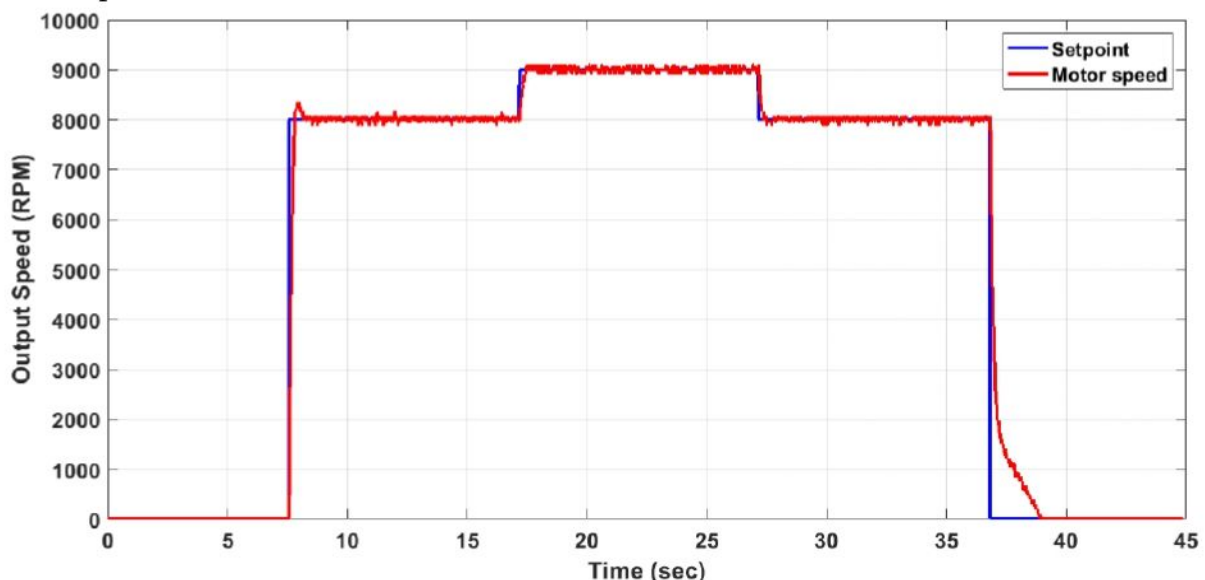


Fig: Closed-loop step response with designed PI controller

Conclusions:

L6b: Position control of DC motor using FOLIPD model

Aim: This experiment will model the DC motor for position control. A simple step test is carried out and based on the response; DC motor is approximated as a First Order Lag plus Integral plus Delay (FOLIPD) model and its parameters will be determined consequently. The parameters of the transfer function will be fine-tuned against experimental responses, in order to validate the model.

Theory

A common actuator in control systems is the DC motor. It directly provides rotary motion and, coupled with wheels or drums and cables, can provide translational motion. The position of the motor is the rotation of the motor shaft or the degree of the rotation which is to be controlled by giving the feedback to the controller which rectifies the controlled output to achieve the desired position. A DC position control system is basically an integrating system (i.e.) for a constant input, the system will give a constantly increasing output. Applying a step input to the motor will yield a response of a First Order Lag plus Integral plus Delay (FOLIPD) as:

$$G(s) = \frac{\Delta Y(s)}{\Delta U(s)} = \frac{K e^{-t_d s}}{s(\tau s + 1)} \quad (1)$$

where K is the gain, τ is the time constant and t_d is the dead time.

The step response of the model (7.1) is

$$s(t) = K (t - t_d - \tau (1 - e^{-(t-t_d)/\tau})) \quad (2)$$

The gain K and average residence time $T_{av} = t_d + \tau$ can be determined graphically as shown in figure 1. The dead time L and the time constant T can be determined by fitting the equation (2) to one point of the step response. A suitable point is $s_{(t_d + \tau)} = K \tau e^{-1}$ which gives

$$\tau = \frac{s_{(t_d + \tau)} e^1}{K} \quad (3)$$

By this we have an approximate FOLIPD model for the position control system.

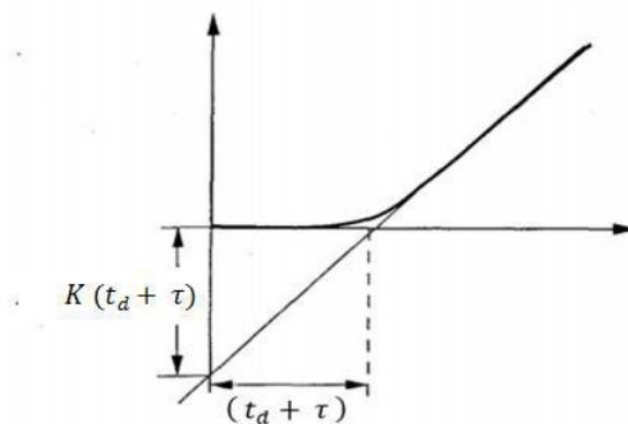


Fig: Graphical determination of a three-parameter model for an integrating process

Procedure

1. The First Order Lag plus Integral plus Delay (FOLIPD) model is given by:

$$G(s) = \frac{\Delta Y(s)}{\Delta U(s)} = \frac{Ke^{-t_d s}}{s(\tau s + 1)} \quad (4)$$

2. In the 'open-loop' mode, input $u(t)$ to the motor is in terms of PWM and the output $y(t)$ is the angular position of the shaft of the motor.
3. First, go to the experiment file, right click on it and run the file as an administrator.
4. Then, click on Play button, on the top left corner of the interface
5. Select appropriate COM port to which the USB-serial cable is connected.
6. Put the value of PWM as 20 in PWM control box.
7. Select the sampling time as 4ms in the respective control box and click on the RUN button;
8. The experiment will run until it has reached a specific position, and then the motor will automatically stop, and you will have the respective graphs of input and output.
9. Once the motor stops, a dialogue box showing the file path will be displayed on the interface; (Do not click the STOP button)
10. For this identification procedure normalize input and output values with respect to input.
11. Determine the value of gain K and the average residence time T_{ar} graphically as in Fig
12. Compute the value of step response $s_{(td+\tau)}$ at $(td+\tau)$ time.
13. Calculate time constant and dead time $\tau = \frac{s_{(td+\tau)} e^1}{K}$

Sample results

1. The First Order Lag plus Integral plus Delay (FOLIPD) model is given by:

$$G(s) = \frac{\Delta Y(s)}{\Delta U(s)} = \frac{Ke^{-t_d s}}{s(\tau s + 1)}$$

We have to find the parameters: gain K , time constant τ and dead time t_d .

2. The step response plot is given in Fig.7.2. From the plot, the following are noted

$$K(t_d + \tau) = 0.8783 ; t_d + \tau = 0.0192 ; K = \frac{0.8783}{0.0192} = 45.744$$

3. The step response value at $(t_d + \tau)$ time is noted $s_{(td+\tau)} = 0.23$

4. Therefore we get the value of time constant τ as

$$\tau = \frac{0.23 * e^1}{45.744} ; \tau = 0.0137$$

5. Dead time t_d is $t_d + \tau = 0.0779 ; t_d = 0.0055$

6. Therefore the transfer function of the position control system is

$$G(s) = \frac{45.744e^{-0.0055s}}{s(0.0137s + 1)}$$

7. The obtained model is validated against the plant response by applying same step input to the model and comparing responses of the plant and the model. The responses of model and plant are shown in Fig. 3. It is clearly seen that obtained model fairly represents the system.

PID Control Gains for Motor Position Control

Aim: The objective of this experiment is to design a closed-loop control system that regulates position of the DC motor.

Theory

Position control is in some sense more difficult than velocity control, because it causes system to go unstable easily due to the presence of an additional integrator in the transfer

function. The goal of this experiment is for the angle of the disk load that is connected to the motor shaft which is to track a user commanded angular position. To control the position of a

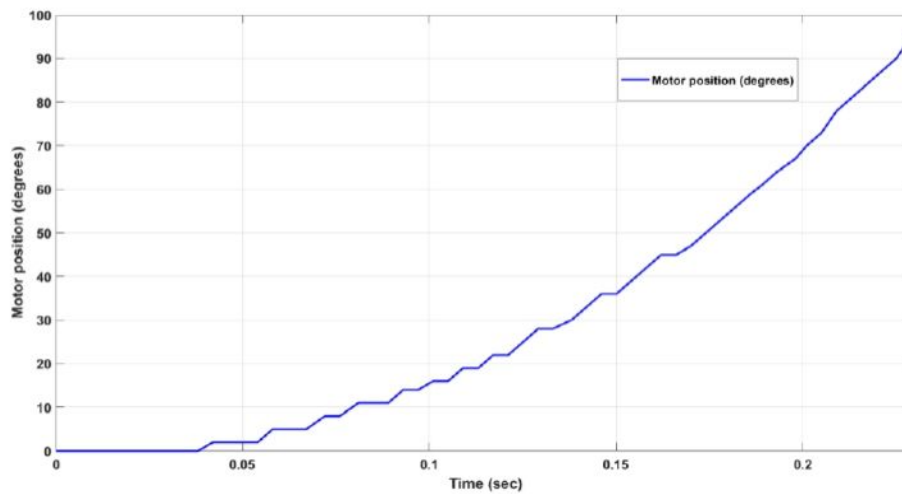


Fig: Step response

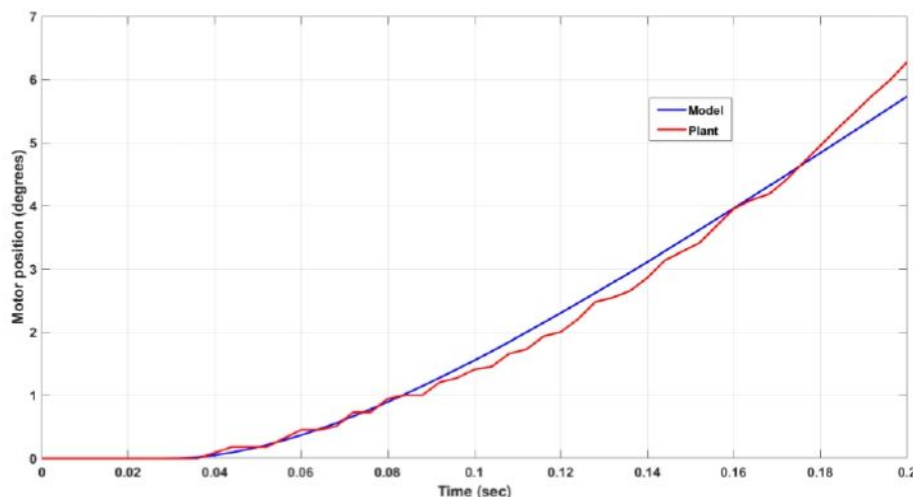


Fig: Model validation

motor shaft, the simplest strategy is to use a PID controller which has position error as the input. The FOLIPD model obtained in experiment 7a is the mathematical representation of the DC motor shaft position and is thus used to design the PID controller. In this experiment, a PID tuning rule by Astrom and Hagglund given in the book: Handbook of PI and PID Controller Tuning Rules by A. O'Dwyer (Imperial College Press UK, 2009) is implemented. This method is suitable for FOLIPD systems. The controller parameters are calculated using the parameters like gain (K_p), time constant τ and dead time (t_d) obtained in earlier experiment

Procedure

1. Firstly, go to the experiment file, right click on it and run the file as an administrator.
2. Then, click on Play button, on the top left corner of the interface
3. If the file is running the Play button will be as shown below;
4. Select appropriate COM port to which USB-serial cable is connected and click on RUN.
5. For sample PI values, click on the Sample PID values button;
6. Since we are now working in the closed-loop configuration, the operating point is reached by giving a set point in degrees.

7. Give a new appropriate set point in degrees and observe that motor position tracks the new set point and settles there. The set point can be varied from the Set Point control.
9. The PID controller placed in the forward path is to be designed and implemented as

$$G(s) = K_c \left(1 + \frac{1}{T_i s} + T_d s \right) = \frac{Y(s)}{U(s)} \quad (5)$$

where, $e(t) = y_{set}(t) - y(t)$ is the error signal and $u(t)$ is the controller output fed to the plant.

10. Using the model obtained in the previous experiment, the parameter K_c , T_i and T_d of PID controller have to be calculated.

11. A brief overview of the Astrom Hagglund method for FOLIPD system is presented is here. The PID gains are computed as

$$K_c = \frac{1}{K_p t_d} \left[0.37 + 0.02 \frac{\tau}{t_d} \right]; T_i = \frac{\left[0.37 + 0.02 \frac{\tau}{t_d} \right]}{\left[0.03 + 0.0012 \frac{\tau}{t_d} \right]} t_d; T_d = \frac{\left[0.16 + 0.28 \frac{\tau}{t_d} \right]}{\left[0.37 + 0.02 \frac{\tau}{t_d} \right]} t_d$$

12. Enter the values of K_c , T_i and T_d (in minutes) you have calculated in the window labeled K_c , T_i and T_d respectively.

13. Perform closed loop test.

14. Give a set point 180 degrees and then 360 degrees and observe whether the motor tracks the new set point.

15. To check the controller performance, a negative step of back to 180 degrees and then to 0 degrees is set to see if the motor position changes accordingly and reaches the new set point.

Sample Results

1. We have used a PID tuning rule by Astrom and Hagglund given in the book: Handbook of PI and PID Controller Tuning Rules by A. O'Dwyer (Imperial College Press UK, 2009).

2. Using these relations, we got $K_c = 1.6686$, $T_i = 0.0012$ minutes and $T_d = 0.0001872$ minutes. These values were used for the implementation of PID controller.

3. Give a set point 180 degrees and then 360 degrees and observe whether the motor tracks the new set point.

4. To check the controller performance, a negative step of back to 180 degrees and then to 0 degrees is set to see if the motor position changes accordingly and reaches the new set point.

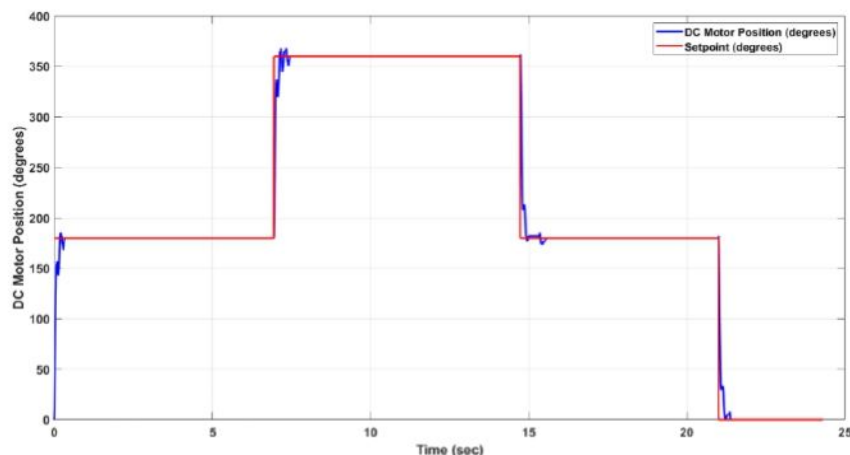


Figure: Closed-loop response with designed PID controller

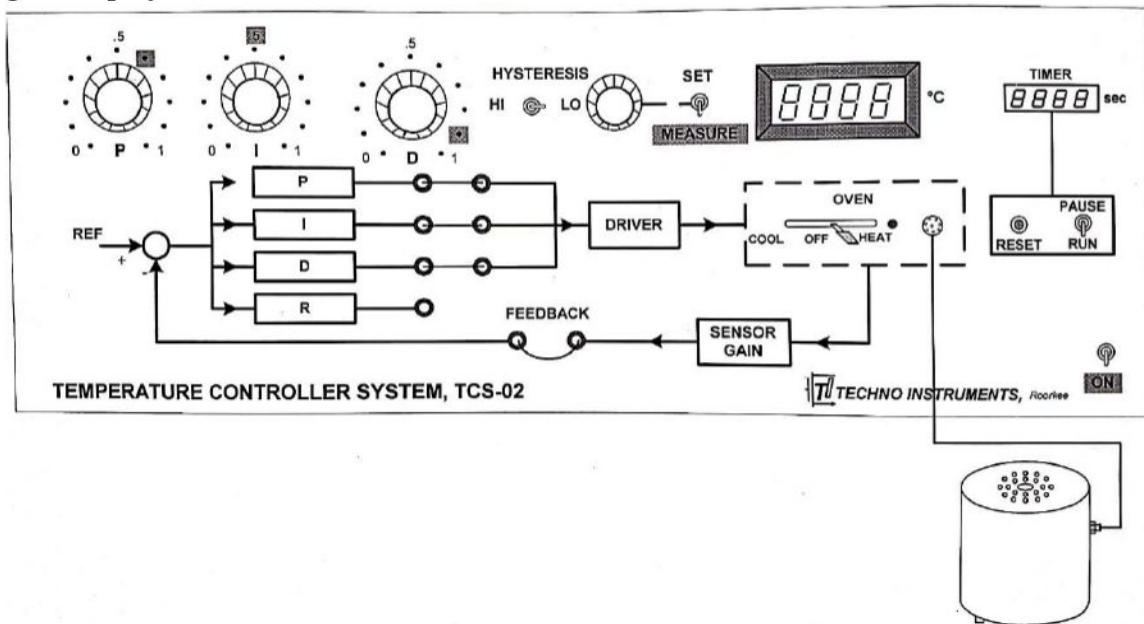
Conclusions:

L7: Oven parameter identification and temperature control

Aim: To identify the oven parameters using reaction curve method and to tune P, PI and PID parameters for temperature control.

Description of trainer kit:

The plant to be controlled is a specially designed oven having a short heating as well as cooling time. The temperature time data may be obtained manually. A solid state temperature sensor converts the absolute temperature information to a proportional electric signal. The reference and actual temperatures are indicated in degree Celsius on a switch selectable digital display.



Identification of Oven Parameters

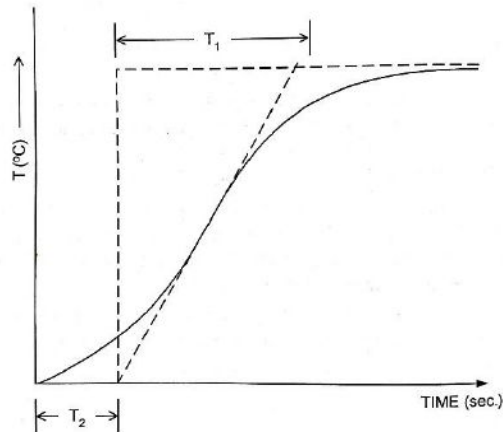
Plant identification is the first step before an attempt can be made to control it. In the present case, the oven equations are obtained experimentally from its step response as outlined below. The procedure is as per reaction curve mention.

In the open-loop testing, the oven is driven through the P – amplifier set to a gain of 10. The input to this amplifier is adjusted through reference potentiometer (the one next to switch S2). This input can be seen on digital display, so that when you set 5°C, the input to proportional amplifier is 50mv (@10mV/ °C) and its output (which acts as input to driver circuit is 0.5V.

Procedure:

1. Keep switch S1 to “Pause”, S2 to “SET” and open ‘Feedback’ terminals.
2. Connect the P output to the driver input and Switch ON the unit.
3. Set P potentiometer to 0.5 which gives $K_p = 10$. Adjust reference potentiometer to read 5.0 on the DVM. This provides an input of 0.5 V to the driver.
4. Put switch S2 to the “Measure” position and note temperature every 10 sec., till the temperature becomes almost constant. Use the timer on the panel to monitor time.
5. Plot temperature – time curve on a graph paper, calculate T1 and T2 and hence write the transfer function of the oven including its driver as:

$$G(s) = \frac{K \exp(-sT_2)}{(1 + sT_1)}$$



Proportional Controller

The value of K_p for P – controller is given as:

$$K_p = \frac{1}{K} \times \frac{T_1}{T_2} (\%)$$

Procedure:

1. Starting with a cool oven, keep switch S1 to “Pause” position and connect P output to the driver input. Keep R, D and I outputs disconnected. Short “Feedback” terminals.
2. Set P potentiometer to the above calculated value of K_p , keeping in mind that the maximum gain is 20.
3. The measurement and interpretation of K_p and P – Control potentiometer setting need some explanation here. the formulae for K_p above is for a unity feedback system and has dimension volts/ °C. In the present unit a temperature sensor having a sensitivity of 10mV/°C (0.01 V/°C) is used between oven output and controller input. Thus the K_p calculated above will need to be divided by 0.01 to obtain the P-Control potentiometer setting.

$$K_{pset} = \frac{K_p}{20 * 0.01}$$

4. Select and set the desired temperature to say 60°C.
5. Keep switch S1 to “Run” position and record temperature readings.
6. Plot the observations on a linear graph and observe the rise time, steady state error and percent overshoot.

PI Controller

The value of K_p and K_I for P-I controller is given as:

$$K_p = \frac{0.9}{K} \times \frac{T_1}{T_2} (\%); K_I = \frac{1}{3.3T_2}$$

Procedure

1. Starting with a cool oven, keep switch S1 to “Pause”, connect P and I outputs to driver input and disconnect R and D outputs. Short feedback terminals.

2. Set P and I potentiometers to the above values of K_P and K_I respectively, keeping in mind that the maximum value of K_P is 20 and that of K_I is 0.036.

$$K_{Iset} = \frac{K_I * 100}{0.036}$$

3. Select and set the desired temperature to say 60°C.
4. Keep switch S1 to "Run" position and record temperature readings.
5. Plot the response on a graph paper and observe the steady state error and percentage overshoot.

PID Controller

The value of K_P , K_I and K_D for PID controller is given as:

$$K_P = \frac{1.2}{K} \times \frac{T_1}{T_2} (\%); K_I = \frac{1}{2T_2}; K_D = 0.5T_2$$

Procedure

1. Starting with a cool oven, keep switch S1 to "Pause", connect P, I and D outputs to driver input and disconnect R output. Short feedback terminals.
2. Set P, I and D potentiometers to the above values of K_P , K_I and K_D respectively, keeping in mind that the maximum value of K_P is 20, K_I is 0.036 and $K_D = 23.5$.

$$K_{Dset} = \frac{K_D * 100}{23.5}$$

3. Select and set the desired temperature to say 60°C.
4. Keep switch S1 to "Run" position and record temperature-time readings.
5. Plot the response on a graph paper and observe the steady state error and percentage overshoot.

Conclusions:

L8a: Lag compensator design

Aim: To design implement and study the effects of LAG compensation network for a given system for a steady state error = 5% and Phase Margin = 40° .

Description: The experiment has been designed with objective of exposing the learner with the problem of control system compensation. A simulated system of 'unknown dynamics' is available which can be studied in time as well as frequency domain. Before design of a particular compensator on a given system, bode plot needs to be designed by using pen & paper. The analog kit consists of three knobs for providing input signals: sine wave, square wave and trigger signal whose frequencies can be varied. The kit also consists of an uncompensated plant, error detector-cum-gain block, a compensation circuit and a power supply module. There is an additional network block where the user can insert resistances/capacitances needed for the design of lag/lead compensator.

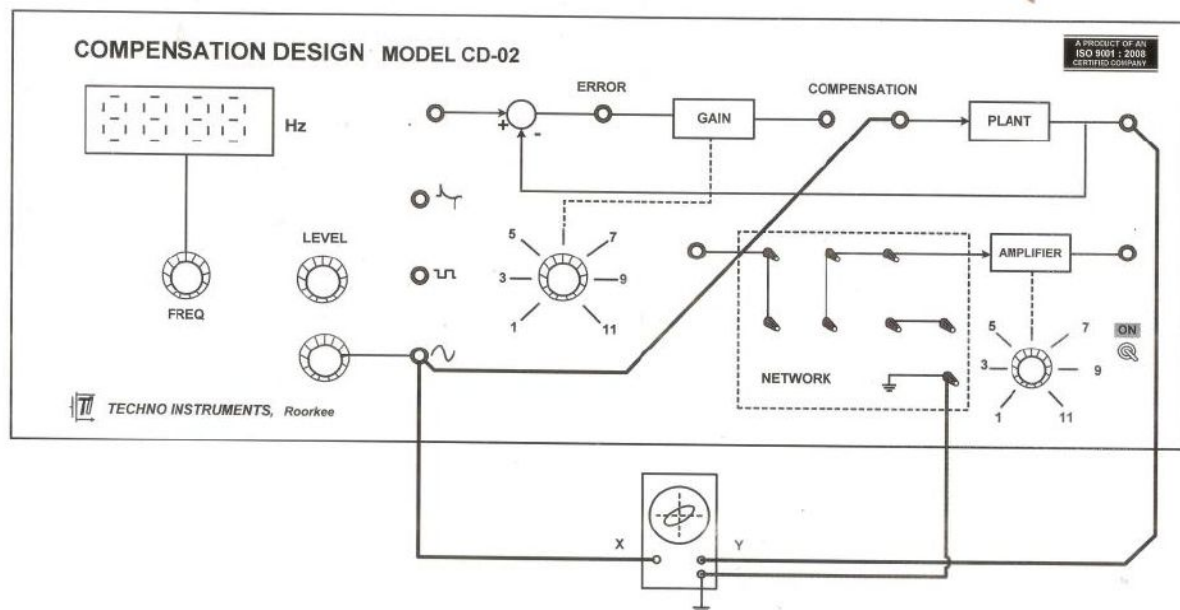


Fig: Open loop module for bode plot analysis

Module 1: Determining the Bode plot of the plant

The first step in any compensator design requires determining the transfer function of the plant by either time domain response or frequency domain response. In the present work, the bode plot analysis is carried out to determine the transfer function. The steps involved are:

- 1) Disconnect the COMPENSATION terminals and apply a constant input of 1 V p-p to the plant from the in-built sine wave source as seen in figure.
- 2) Once the input signal is steady, put the CRO in X-Y mode for the measurements to be carried out. Vary the frequency in steps and calculate the plant gain in dB and calculate the phase angle in degree at each frequency as seen in table.
- 3) From low frequency end of magnitude plot, obtain error coefficient and the steady state error. Also, calculate the forward path gain K to meet the steady state specifications.

- 4) Measurements are carried out by double trace CRO. If the input $x = A \cos(\omega t)$ and output $y = B \cos(\omega t - \theta)$ are fed to input and output plates of a CRO, the resulting trace is an ellipse and the phase, magnitude can be calculated, as observed in the below Fig, as:

$$\text{Gain} = B/A = y_0/x_0 \text{ or } 20 \log (B/A) \text{ Db}$$

$$\text{Phase } \theta = -\sin^{-1}(x_0/A) = -\sin^{-1}(y_0/B)$$

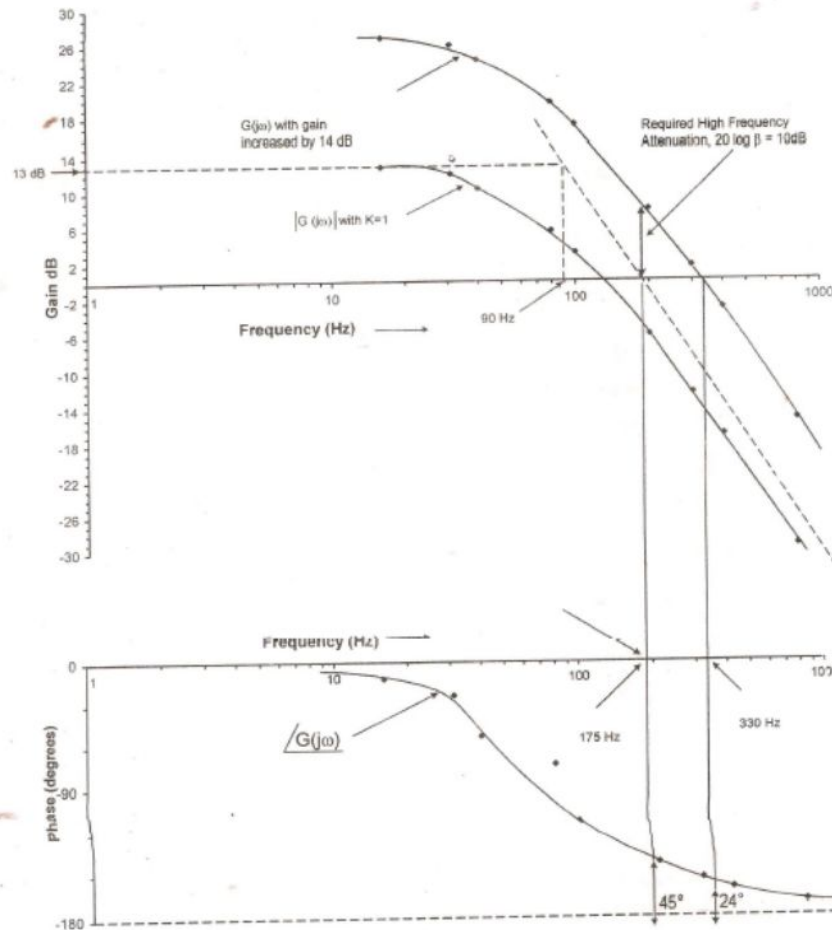


Fig: Bode plot

- 5) From the table readings, plot the values of magnitude and phase in Bode plot. From the bode plot the transfer function is approximated by drawing the low and high frequency asymptotes, observing the values of low frequency gain and corner frequency.

Corner frequency $f_r =$ _____ yields $T = 1 / 2\pi f_r =$ _____

Gain at low frequency $G =$ _____ yields $K_1 = \text{antilog}(G/20) =$ _____

Thus the plant transfer function is $K_1/(1+T)^2$ (Note high frequency slope is -40db/sec)

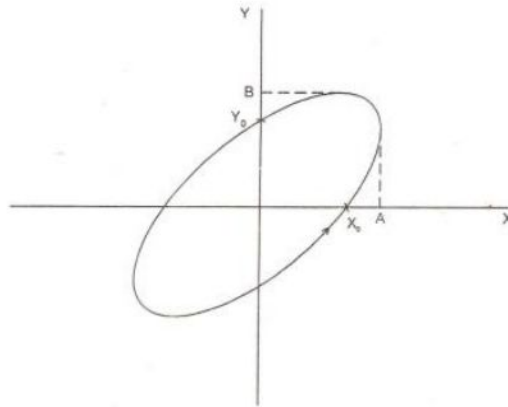


Fig: Phase and gain measurement from CRO

Freq	A	B	x_0	y_0	Gain (dB)	Phase (Deg)

Module 2: Lag Compensator design

The steps involved for the lag compensator are:

- 1) From the Bode plot, find a frequency where $PM_{actual} = PM_{specified} + \text{safety margin } (5^\circ)$. This is the new cross over frequency $w_{g,new}$
- 2) Measure the gain at $w_{g,new}$. This must equal the high frequency attenuation of the lag network i.e $20 \log \beta$. Compute β .
- 3) Choose $Z_c = 1/T$ at approximately $0.1 * w_{g,new}$ and $P_c = 1/\beta T$ accordingly.
- 4) Write the transfer function and calculate R_1 , R_2 and C .
- 5) Implement new transfer function with the help of passive components and amplifier provided for the purpose. Amplifier gain has to be set to unity.
- 6) Insert compensator and find the phase margin of the plant.

The given design requirements are : Steady state error = 5% and Phase Margin = 40°

The required value error coefficient K_p to meet the steady state specifications is 19. Hence the gain K needs to be increased by $19/ K_1$ or $20 \log(19/ K_1) = \underline{\hspace{2cm}}$ dB. Now, the magnitude plot is drawn on Bode plot shifting up by $\underline{\hspace{2cm}}$ dB which results in:

Gain cross over frequency w_g : $\underline{\hspace{2cm}}$

Phase Margin (ϕ_1): $\underline{\hspace{2cm}}$

The phase margin needed = $40^\circ + 5^\circ = 45^\circ$

This is available naturally at $\omega_{g,new} = \omega_g / 10 =$ _____

The high frequency attenuation needed is = $10\text{dB} = 20\log \beta$ so that the value of β is 3.16. The design of the compensation network is thus chosen as:

$$Z_c = 1/T = 2\pi \omega_{g,new} = \text{_____,} \quad P_c = 1/\beta T = \text{_____}$$

$$T = R_2 C = 0.009094 \quad \& \quad \beta = (R_1 + R_2)/R_2 = 3.16$$

Solving the above equations, approximate values of the three components are:

$R_1 \sim 20\text{k}\Omega$; $R_2 \sim 9.1\text{k}\Omega$; $C \sim 1\mu\text{F}$.

Conclusions:

L8b: Lead compensator design

Aim: To design implement and study the effects of LEAD compensation network for a given system with steady state error = 5% and Phase Margin = 40° .

Module 1: Determining the Bode plot of the plant

The steps are similar to the steps for determining Bode plot in LAG compensator experiment.

- 1) From the table readings, plot the values of magnitude and phase in Bode plot. From the bode plot the transfer function is approximated by drawing the low and high frequency asymptotes, observing the values of low frequency gain and corner frequency (fig 10.2)

Corner frequency $f_r = \underline{\hspace{2cm}}$ yields $T = 1 / 2\pi f_r = \underline{\hspace{2cm}}$

Gain at low frequency $G = \underline{\hspace{2cm}}$ yields $K_1 = \text{antilog}(G/20) = \underline{\hspace{2cm}}$

Thus the plant transfer function is $K_1/(1+T)^2$ (Note high frequency slope is -40db/sec)

Freq	A	B	x_0	y_0	Gain (dB)	Phase (Deg)

Module 2: Lead Compensator design

The steps involved in design of a lead compensator are:

- 1) The phase lead (ϕ_m) = $PM_{\text{specified}} - PM_{\text{available}} + \text{safety margin}$ (5° to 10°).
- 2) Calculate α for the lead network using $\alpha = (1 - \sin \phi_m) / (1 + \sin \phi_m)$.
- 3) Calculate new gain cross over frequency $w_{g,\text{new}}$ such that gain G at $w_{g,\text{new}} = 10 \log \alpha$.
- 4) The corner frequencies are calculated from $1/T = \sqrt{\alpha} w_m$ and $1/\alpha T = w_m / \sqrt{\alpha}$.
- 5) Implement the transfer function with the help of few passive components and amplifier which should be set to a gain equal to $1/\alpha$.
- 6) Insert compensator and find the phase margin of the plant.

The given design requirements are: Steady state error = 5% and Phase Margin = 40°

The gain setting of 5 has to be kept constant throughout the experiment.

Phase lead $\phi_m = 40^\circ - \phi_1 + 10^\circ =$ _____

The value of $\alpha = (1 - \sin \phi_m) / (1 + \sin \phi_m) =$ _____

G at $w_{g, \text{new}} = 10 \log \alpha$ corresponds to $w_m = 2 \cdot \pi \cdot$ _____ = _____

$1/T = \sqrt{\alpha} w_m =$ _____ and $1/\alpha T = w_m / \sqrt{\alpha} =$ _____

Time constant $T = R_1 C$. Choosing the value of $C = 0.01 \mu\text{F}$, resistances computed are:

$R_1 = T/C =$ _____ Ω ; $R_2 = \alpha R_1 / (1 - \alpha) =$ _____ Ω

Conclusions:

L9: DSP Hardware Experiments 1: Signal Processing

Aim: To implement basic DSP algorithms using TMS320C6748

1. Write a C program to perform linear convolution of two sequences.

```
#include<stdio.h>
#define LENGHT1 6 /*Lenght of i/p samples sequence*/
#define LENGHT2 4 /*Lenght of impulse response Co-efficients */
int x[2*LENGHT1-1]={1,2,3,4,5,6,0,0,0,0,0}; /*Input Signal Samples*/
int h[2*LENGHT1-1]={1,2,3,4,0,0,0,0,0,0,0}; /*Impulse Response Coefficients*/
int y[LENGHT1+LENGHT2-1];
main()
{
    int i=0,j;
    for(i=0;i<(LENGHT1+LENGHT2-1);i++)
    {
        y[i]=0;
        for(j=0;j<=i;j++)
            y[i]+=x[j]*h[i-j];
    }
    for(i=0;i<(LENGHT1+LENGHT2-1);i++)
        printf("%d\n",y[i]);}
```

2. Write a C program to perform circular convolution of two sequences.

```
#include<stdio.h>
int m,n,x[30],h[30],y[30],i,j,temp[30],k,x2[30],a[30];
void main()
{

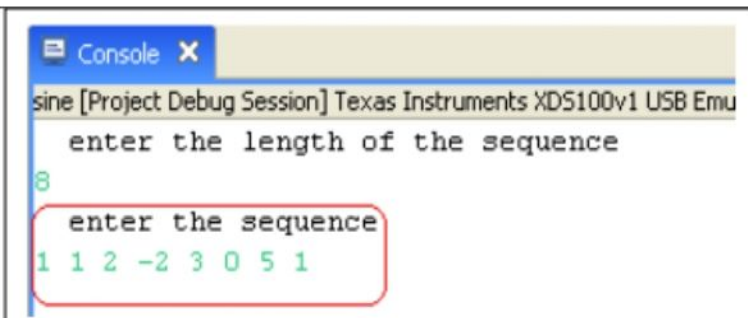
    printf(" enter the length of the first sequence\n");
    scanf("%d",&m);
    printf(" enter the length of the second sequence\n");
    scanf("%d",&n);
    printf(" enter the first sequence\n");
    for(i=0;i<m;i++)
        scanf("%d",&x[i]);
    printf(" enter the second sequence\n");
    for(j=0;j<n;j++)
        scanf("%d",&h[j]);
    if(m-n!=0) /*If lenght of both sequences are not equal*/
    {
        if(m>n) /* Pad the smaller sequence with
zero*/
        {
            for(i=n;i<m;i++)
                h[i]=0;
            n=m;
        }
    }
}
```

3. Write a C program to compute DFT of a given sequence $x(n)$.

```
#include<stdio.h>
#include<math.h>
int N,k,n,i;
float pi=3.1416,sumre=0, sumim=0,out_real[8]={0.0}, out_imag[8]={0.0};
int x[32];
void main(void)
{
printf("enter the length of the sequence\n");
scanf("%d",&N);
printf(" enter the sequence\n");
for(i=0;i<N;i++)
scanf("%d",&x[i]);
for(k=0;k<N;k++)
{
sumre=0;
sumim=0;

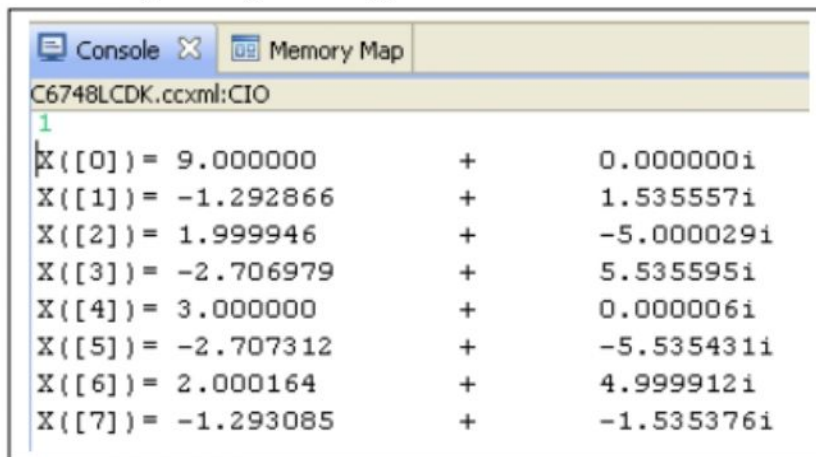
for(n=0;n<N;n++)
{
sumre=sumre+x[n]* cos(2*pi*k*n/N);
sumim=sumim-x[n]* sin(2*pi*k*n/N);
}

out_real[k]=sumre;
out_imag[k]=sumim;
printf("X([%d])=\t%f\t+\t%fi\n",k,out_real[k],out_imag[k]);
}
}
```



```
Console X
[Project Debug Session] Texas Instruments XDS100v1 USB Emu
enter the length of the sequence
8
enter the sequence
1 1 2 -2 3 0 5 1
```

DFT of the given input will appear in console window as shown below



```
C6748LCDK.ccxml: CIO
1
X([0]) = 9.000000      +      0.000000i
X([1]) = -1.292866    +      1.535557i
X([2]) = 1.999946     +     -5.000029i
X([3]) = -2.706979    +      5.535595i
X([4]) = 3.000000     +      0.000006i
X([5]) = -2.707312    +     -5.535431i
X([6]) = 2.000164     +      4.999912i
X([7]) = -1.293085    +     -1.535376i
```

4. Write a C program to compute IDFT of a given sequence X(K).

```
#include<stdio.h>
#include<math.h>
#define pi 3.14

void main()
{
float xr[20],xi[20],y[20];
int N,i,k,n,theta;

printf("\nEnter the no of points of idft \t:");
scanf("%d",&N);
printf("\nEnter the function variables of f(n) to calculate IDFT in the complex
form (e.g. 3+j4 enter as a 3 4)\n");
for(i=0;i<N;i++)
{
scanf("%f\t%f",&xr[i],&xi[i]);
}

for(k=0;k<N;k++)
{
y[k]=0.0;

for(n=0;n<N;n++) /*IDFT calculation*/
{
theta= (2*pi*k*n)/N;
y[k]=y[k]+xr[n]*cos(theta)-xi[n]*sin(theta);
}
y[k]=y[k]/N;
}
/*Display values of F[k]*/
printf("\nIDFT OF GIVEN SIGNAL:-\n");
for(i=0;i<N;i++)
{
printf("\n%f\n",y[i]);
}
}
```

5. Write a c program to compute N-point DTF using FFT and plot the spectrum of a given signal.

```
#include <math.h>
#define PTS 64 // # of points for FFT
#define PI 3.14159265358979
typedef struct {float real,imag;} COMPLEX;
void FFT(COMPLEX *Y, int n); //FFT prototype
float iobuffer[PTS]; //as input and output buffer
float x1[PTS]; //intermediate buffer
short i; //general purpose index variable
short buffercount = 0; //number of new samples in iobuffer
short flag = 0; //set to 1 by ISR when iobuffer full
COMPLEX w[PTS]; //twiddle constants stored in w
COMPLEX samples[PTS]; //primary working buffer
main()
{
    for (i = 0 ; i < PTS ; i++) // set up twiddle constants in w
    {
        w[i].real = cos(2*PI*i/(PTS*2.0)); //Re component of twiddle constants
        w[i].imag = -sin(2*PI*i/(PTS*2.0)); //Im component of twiddle constants
    }
    for (i = 0 ; i < PTS ; i++) //swap buffers
    {
        iobuffer[i] = sin(2*PI*10*i/64.0); /*10 -> freq, 64 -> sampling freq*/
        samples[i].real=0.0;
        samples[i].imag=0.0;
    }
    for (i = 0 ; i < PTS ; i++) //swap buffers
    {
        samples[i].real=iobuffer[i]; //buffer with new data
    }
    for (i = 0 ; i < PTS ; i++)
        samples[i].imag = 0.0; //imag components = 0
    FFT(samples,PTS); //call function FFT.c
    for (i = 0 ; i < PTS ; i++) //compute magnitude
    {
        x1[i] = sqrt(samples[i].real*samples[i].real+ samples[i].imag*samples[i].imag);
    }
} //end of main
```

```

void FFT(COMPLEX *Y, int N) //input sample array, # of points
{
    COMPLEX temp1,temp2; //temporary storage variables
    int i,j,k; //loop counter variables
    int upper_leg, lower_leg; //index of upper/lower butterfly leg
    int leg_diff; //difference between upper/lower leg
    int num_stages = 0; //number of FFT stages (iterations)
    int index, step; //index/step through twiddle constant
    i = 1; //log(base2) of N points= # of stages
    do
    {
        num_stages +=1;
        i = i*2;
    }while (i!=N);
    leg_diff = N/2; //difference between upper&lower legs
    step = (PTS*2)/N; //step between values in twiddle
    for (i = 0; i < num_stages; i++) //for N-point FFT
    {
        index = 0;
        for (j = 0; j < leg_diff; j++)
        {
            for (upper_leg = j; upper_leg < N; upper_leg += (2*leg_diff))
            {
                lower_leg = upper_leg+leg_diff;
                temp1.real = (Y[upper_leg]).real + (Y[lower_leg]).real;
                temp1.imag = (Y[upper_leg]).imag + (Y[lower_leg]).imag;
                temp2.real = (Y[upper_leg]).real - (Y[lower_leg]).real;
                temp2.imag = (Y[upper_leg]).imag - (Y[lower_leg]).imag;
                (Y[lower_leg]).real = temp2.real*(w[index]).real
                -temp2.imag*(w[index]).imag;
                (Y[lower_leg]).imag = temp2.real*(w[index]).imag
                +temp2.imag*(w[index]).real;
                (Y[upper_leg]).real = temp1.real;
                (Y[upper_leg]).imag = temp1.imag;
            }
            index += step;
        }
        leg_diff = leg_diff/2;
        step *= 2;
    }
    j = 0;
    for (i = 1; i < (N-1); i++) //bit reversal for resequencing data
    {
        k = N/2;
        while (k <= j)
        {
            j = j - k;
            k = k/2;
        }
        j = j + k;
        if (i<j)
        {
            temp1.real = (Y[j]).real;
            temp1.imag = (Y[j]).imag;
            (Y[j]).real = (Y[i]).real;
            (Y[j]).imag = (Y[i]).imag;
            (Y[i]).real = temp1.real;
            (Y[i]).imag = temp1.imag;
        }
    }
}

```

```

}
}
return;
}

```

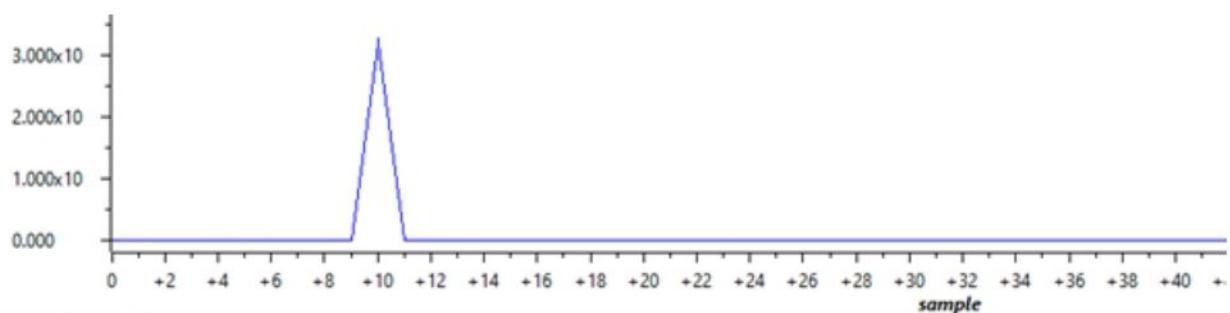
Result: After debug, Observe the spectrum using the following steps.

- Run the program to see the results.
- To view results, use Graph Tools.
- Set the graph properties as shown in figure.

Graph Properties

Property	Value
▼ Data Properties	
Acquisition Buffer Size	128
Dsp Data Type	32 bit floating point
Index Increment	1
Q Value	0
Sampling Rate Hz	1
Start Address	x1
▼ Display Properties	
Auto Scale	☒ true
Axis Display	☒ true
Data Plot Style	Line
Display Data Size	64
Grid Style	No Grid
Magnitude Display Scale	Linear
Time Display Unit	sample
Use Dc Value For Graph	☐ false

Import Export OK Cancel



Exercise:

1. Write a C program to suppress the Bass from a given audio file using appropriate FIR filter (input to the target board is music from mobile phone).

2. Write a c program to compute Power Spectral Density (PSD) of a signal.

The value of the auto-correlation function at zero-time equals the total power in the signal. To compute PSD, We compute the auto-correlation of the signal and then take its FFT.

ALGORITHM TO IMPLEMENT PSD:

- Step 1 - Select no. of points for FFT(Eg: 128)
- Step 2 – Generate a sine wave of frequency ‘f’ (eg: 5 Hz with a sampling rate = No. of Points of FFT (eg. 128)) using math library function.
- Step 3 - Compute the Auto Correlation of Sine wave
- Step4 - Take output of auto correlation, apply FFT algorithm. (Use Graph option to view the PSD).
- Step 5 - Repeat Step-1 to 4 for different no. of points & frequencies.

‘C’ Program to Implement PSD:

```
#include <math.h>
#define PTS 128 // # of points for FFT
#define PI 3.14159265358979
typedef struct {float real,imag;} COMPLEX;
void FFT(COMPLEX *Y, int n); //FFT prototype
float iobuffer[PTS]; //as input and output buffer
float x1[PTS],x[PTS]; //intermediate buffer
short i; //general purpose index variable
short buffercount = 0; //number of new samples in iobuffer
short flag = 0; //set to 1 by ISR when iobuffer full
float y[128];
COMPLEX w[PTS]; //twiddle constants stored in w
COMPLEX samples[PTS]; //primary working buffer
main()
{
    float j,sum=0.0 ;
    int n,k,i,a;
    for (i = 0 ; i<PTS ; i++) // set up twiddle constants in w
    {
        w[i].real = cos(2*PI*i/(PTS*2.0));
        /*Re component of twiddle constants*/
        w[i].imag = -sin(2*PI*i/(PTS*2.0));
        /*Im component of twiddle constants*/
    }

    /*****Input Signal X(n) *****/
    for(i=0,j=0;i<PTS;i++)
    { x[i] = sin(2*PI*5*i/PTS);
      // Signal x(Fs)=sin(2*pi*f*i/Fs);
      samples[i].real=0.0;

      samples[i].imag=0.0;
    }
    /*****Auto Correlation of X(n)=R(t) *****/
    for(n=0;n<PTS;n++)
    {
        sum=0;
        for(k=0;k<PTS-n;k++)
        {
            sum=sum+(x[k]*x[n+k]); // Auto Correlation R(t)
        }
    }
```

```

iobuffer[n] = sum;
}
/***** FFT of R(t) *****/
for (i = 0 ; i < PTS ; i++) //swap buffers
{
samples[i].real=iobuffer[i]; //buffer with new data
}
for (i = 0 ; i < PTS ; i++)
samples[i].imag = 0.0; //imag components = 0
FFT(samples,PTS); //call function FFT.c
/***** PSD *****/
for (i = 0 ; i < PTS ; i++) //compute magnitude
{
x1[i] = sqrt(samples[i].real*samples[i].real
+ samples[i].imag*samples[i].imag);
}
} //end of main

/*FFT*/

/*
#define PTS 128 //# of points for FFT
typedef struct {float real,imag;} COMPLEX;
extern COMPLEX w[PTS]; //twiddle constants stored in w*/
void FFT(COMPLEX *Y, int N) //input sample array, # of points
{
COMPLEX temp1,temp2; //temporary storage variables
int i,j,k; //loop counter variables
int upper_leg, lower_leg; //index of upper/lower butterfly leg
int leg_diff; //difference between upper/lower leg
int num_stages = 0; //number of FFT stages (iterations)

int index, step; //index/step through twiddle constant
i = 1; //log(base2) of N points= # of stages
do
{
num_stages +=1;
i = i*2;
}while (i!=N);
leg_diff = N/2; //difference between upper&lower legs
step = (PTS*2)/N; //step between values in twiddle.h// 512
for (i = 0;i < num_stages; i++) //for N-point FFT
{
index = 0;
for (j = 0; j < leg_diff; j++)
{
for (upper_leg = j; upper_leg < N; upper_leg += (2*leg_diff))
{
lower_leg = upper_leg+leg_diff;
temp1.real = (Y[upper_leg]).real + (Y[lower_leg]).real;
temp1.imag = (Y[upper_leg]).imag + (Y[lower_leg]).imag;
temp2.real = (Y[upper_leg]).real - (Y[lower_leg]).real;
temp2.imag = (Y[upper_leg]).imag - (Y[lower_leg]).imag;
(Y[lower_leg]).real = temp2.real*(w[index]).real
-temp2.imag*(w[index]).imag;
(Y[lower_leg]).imag = temp2.real*(w[index]).imag
+temp2.imag*(w[index]).real;
(Y[upper_leg]).real = temp1.real;
(Y[upper_leg]).imag = temp1.imag;

```

```

}
index += step;
}
leg_diff = leg_diff/2;
step *= 2;
}
j = 0;
for (i = 1; i < (N-1); i++)
//bit reversal for resequencing data
{
k = N/2;
while (k <= j)
{
j = j - k;
k = k/2;
}
j = j + k;
if (i<j)
{
temp1.real = (Y[j]).real;
temp1.imag = (Y[j]).imag;
(Y[j]).real = (Y[i]).real;
(Y[j]).imag = (Y[i]).imag;
(Y[i]).real = temp1.real;
(Y[i]).imag = temp1.imag;
}
}
return;
}

```

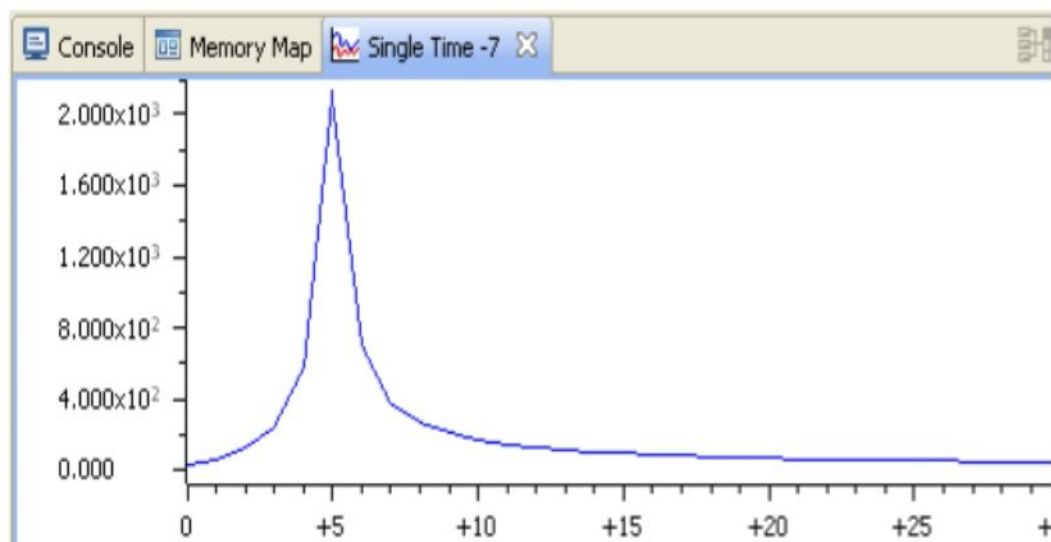
Result: After debug, Observe the spectrum using the following steps.

- Run the program to see the results.
- To view results, use Graph Tools.
- Set the graph properties as shown in figure.

Graph Properties

Property	Value
Data Properties	
Acquisition Buffer Size	64
Dsp Data Type	32 bit floating poin
Index Increment	1
Q_Value	0
Sampling Rate HZ	1
Start Address	x1
Display Properties	
Axis Display	☒ true
Data Plot Style	Line
Display Data Size	64
Grid Style	Major Grid
Magnitude Display Scale	Linear
Time Display Unit	sample
Misc	
Use Dc Value For Graph	☐ false

Import Export OK Cancel



L10: DSP Hardware Experiments 2: FIR and IIR filters

1. Write a C program to implement FIR filter for a given specification using TMS320C6748.

```
#include "L138_LCDK_aic3106_init.h"

#define N 31 //number of coefficients

//float h[N]={-0.2,0.2,-0.2,0.2,-0.2,0.2};

float h[N]={0.01525652688, 0.01155856438, 0.006443395745,-4.29373144e-17,-
0.007614922244, -0.0161819905, -0.02542754449, -0.0350350365, -0.04465886205, -
0.05393996835, -0.06252241135, -0.07007007301, -0.07628263533, -0.08090995252, -
0.08376414329, 0.932015717, -0.08376414329, -0.08090995252, -0.07628263533, -
0.07007007301,-0.06252241135, -0.05393996835, -0.04465886205, -0.0350350365, -
0.02542754449,-0.0161819905,-0.007614922244,-4.29373144e-17, 0.006443395745,
0.01155856438, 0.01525652688}; //HPF-2KHz-Rectangular */

/*float h[N]={-0.01348909363, -0.01021953207,-0.005696943495,2.920240591e-18,
0.006732751615, 0.01430734433, 0.02248182334, 0.03097631037, 0.03948523849,
0.04769114777, 0.05527933314, 0.06195262074, 0.06744547188, 0.0715367198,
0.0740602687, 0.07491308451, 0.0740602687, 0.0715367198, 0.06744547188,
0.06195262074, 0.05527933314, 0.04769114777, 0.03948523849, 0.03097631037,
0.02248182334, 0.01430734433, 0.006732751615,2.920240591e-18,-0.005696943495, -
0.01021953207, -0.01348909363}; //LPF-2KHz-Rectangular*/

/*float h[N]={ 0.02425338328, 0.004938407801,-0.001079704845, 0.02781886607,
0.05735153705, 0.04029640928, -0.01203345321, -0.03613776714,-0.005907028913,
0.01629591547, -0.04440294951, -0.1557321399, -0.1814887673, -0.03456885368,
0.1947017014, 0.3058844209, 0.1947017014, -0.03456885368, -0.1814887673, -
0.1557321399, -0.04440294951, 0.01629591547,-0.005907028913, -0.03613776714, -
0.01203345321, 0.04029640928, 0.05735153705, 0.02781886607,-0.001079704845,
0.004938407801, 0.02425338328}; //BPF-3KHz-10KHz-Rectangular*/

float x[N]; // filter delay line

interrupt void interrupt4(void)
{
    short i;
    float yn = 0.0;

    x[0] = (float)(input_right_sample()); // input from ADC
    for (i=0 ; i<N ; i++) // compute filter output
        yn += h[i]*x[i];
    for (i=(N-1) ; i>0 ; i--) // shift delay line
        x[i] = x[i-1];
    output_right_sample((int16_t)(yn)); // output to DAC
    return;
}

int main(void)
{
    L138_initialise_intr(FS_48000_HZ,ADC_GAIN_0DB,DAC_ATTEN_0DB,LCDK_LINE_INPUT);

    while(1);
} // end of main()
```

Designing filter coefficients using MATLAB

FIR Low pass filters using rectangular, triangular and Kaiser windows

% sampling rate - 8000 Hz

order = 30;

cf--> contains set of cut-off frequencies [Wc]

% cutoff frequency - 500 Hz

b_rect1=fir1(order,cf,boxcar(31)); Rectangular

b_tril=fir1(order,cf,bartlett(31)); Triangular

b_kail=fir1(order,cf,kaiser(31,8)); Kaisar [Where 8-->Beta Co-efficient]

For high filter

b_rect1=fir1(30,0.1,'high',boxcar(31))

Following are the coefficients generated for various cutoff frequencies using different windows

Cutoff -500Hz

float b_rect1[31]={-0.008982,-0.017782,-0.025020,-0.029339,-0.029569,-0.024895, -
0.014970,0.000000,0.019247,0.041491,0.065053,0.088016,0.108421,0.124473,0.134729,
0.138255,0.134729,0.124473,0.108421,0.088016,0.065053,0.041491,0.019247,0.000000, -
0.014970,-0.024895,-0.029569,-0.029339,-0.025020,-0.017782,-0.008982};

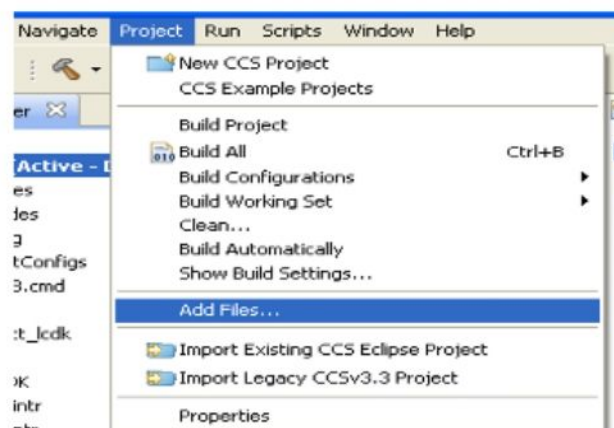
Cutoff -500Hz

float b_kail[31]={-0.000019,-0.000170,-0.000609,-0.001451,-0.002593,-0.003511,
0.003150,0.000000,0.007551,0.020655,0.039383,0.062306,0.086494,0.108031,0.122944,
0.128279,0.122944,0.108031,0.086494,0.062306,0.039383,0.020655,0.007551,0.000000, -
0.003150,-0.003511,-0.002593,-0.001451,-0.000609,-0.000170,-0.000019};

Cutoff -500Hz float b_tril[31]={0.000000,-0.001185,-0.003336,-0.005868,-0.007885,-
0.008298,-0.005988,
0.000000,0.010265,0.024895,0.043368,0.064545,0.086737,0.107877,0.125747,0.138255,
0.125747,0.107877,0.086737,0.064545,0.043368,0.024895,0.010265,0.000000,-0.005988, -
0.008298,-0.007885,-0.005868,-0.003336,-0.001185,0.000000}

Additional steps required to execute the real-time programs

- To add files go to Project-> Add files.

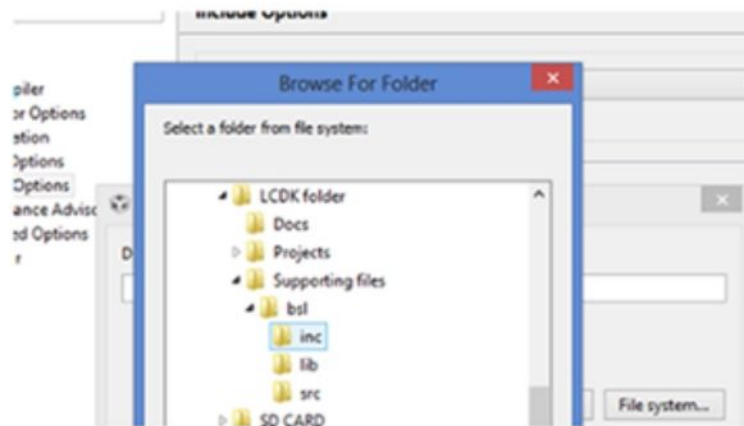
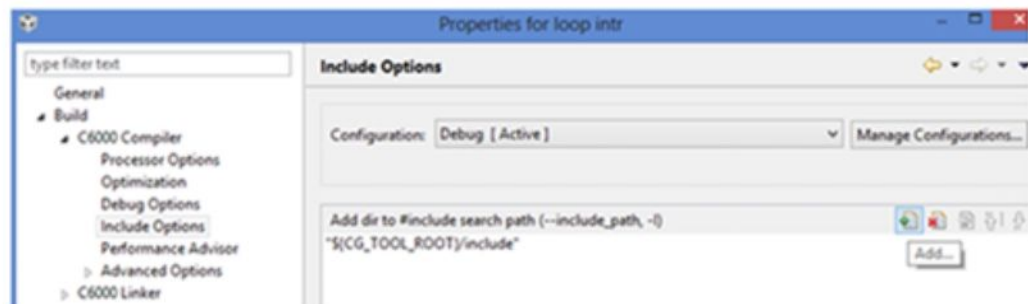


- Follow the path given below to add the required files:

1. Supporting files\evmomapl138_bsl.lib
2. Supporting files \L138_LCDK_aic3106_init.h
3. Supporting files \L138_LCDK_aic3106_init.c
4. Supporting files \vectors_intr.asm

- Right click on the project name and choose "Show Build Settings.." -> Build-> C6000 Compiler-> Include Options -> Click "Add" as shown in the figure.

- Now Click File system -> supporting files->bsl->inc->OK.



- Build Project, Load and Resume the project.

Procedure to check the output

- After successful generation of .out file connect target to host PC
- Connect signal generator to LINE INPUT with 1Vp-p Sine Wave frequency with Frequency of 500Hz
- Connect CRO to LINE OUT with volt/div at 1 and time/div at 5msec
- Vary Frequency of Signal at Function Generator to See the Response (High Pass/Low Pass) according to Selected Filter Coefficients.

2. Write a C program to implement IIR filter for a given specification using TMS320C6748

Designing filter coefficients using MATLAB

```
% IIR Low pass using Butterworth and Chebyshev filters
% sampling rate – 24000 Hz
order = 2;
cf = [2500/12000,8000/12000,1600/12000]; % for different cutoff frequency
% cutoff frequency – 2500 Hz
[num_bw1,den_bw1]=butter(order,cf(1));
[num_cb1,den_cb1]=cheby1(order,3,cf(1));
% cutoff frequency - 8000 Hz
[num_bw2,den_bw2]=butter(order,cf(2));
[num_cb2,den_cb2]=cheby1(order,3,cf(2));
```

IIR_BUTTERWORTH_LP FILTER CO-EFFICIENTS:

Co-Efficients	Fc=2500Hz		Fc=800Hz		Fc=8000Hz	
	Floating Point Values	Fixed Point Values(Q15)	Floating Point Values	Fixed Point Values(Q15)	Floating Point Values	Fixed Point Values(Q15)
B0	0.072231	2366	0.009526	312	0.465153	15241
B1	0.144462	2366[B1/2]	0.019052	312[B1/2]	0.930306	15241[B1/2]
B2	0.072231	2366	0.009526	312	0.465153	15241
A0	1.000000	32767	1.000000	32767	1.000000	32767
A1	-1.109229	-18179[A1/2]	-1.705552,	- 27943[A1/2]	0.620204	10161[A1/2]
A2	0.398152	13046	0.743655	24367	0.240408	7877

Note: We have Multiplied Floating Point Values with 32767(2¹⁵) to get Fixed Point Values.

C program to implement IIR filter

```
#include "L138_LCDK_aic3106_init.h"

const signed int h[] =
{ //12730,-12730,12730,32767,-18324,21137 /*HP 2500 */
  312,312,312,32767,-27943,24367 /*LP 800 */
  //1455,1455,1455,32767,-23140,21735 /*LP 2500 */
  // 9268,-9268,9268,32767,-7395,18367 /*HP 4000*/
  // 7215,-7215,7215,32767,5039,6171, /*HP 7000*/
};

interrupt void interrupt4(void)
{
    static signed int x[6] = { 0, 0, 0, 0, 0, 0 }; /* x(n), x(n-1), x(n-2). Must
be static */
    static signed int y[6] = { 0, 0, 0, 0, 0, 0 }; /* y(n), y(n-1), y(n-2). Must
be static */
    long temp=0;

    temp = input_right_sample(); // input from ADC; /* Copy input to temp */

    x[0] = (signed int) temp; /* Copy input to x[stages][0] */

    temp = ( (long)h[0] * x[0]) ; /* B0 * x(n) */

    temp += ( (long)h[1] * x[1]); /* B1/2 * x(n-1) */
    temp += ( (long)h[1] * x[1]); /* B1/2 * x(n-1) */
    temp += ( (long)h[2] * x[2]); /* B2 * x(n-2) */
    temp -= ( (long)h[4] * y[1]); /* A1/2 * y(n-1) */
    temp -= ( (long)h[4] * y[1]); /* A1/2 * y(n-1) */
    temp -= ( (long)h[5] * y[2]); /* A2 * y(n-2) */

    /* Divide temp by coefficients[A0] */

    temp >>= 15;

    if ( temp > 32767 )
    {
        temp = 32767;
    }
    else if ( temp < -32767)
    {
        temp = -32767;
    }
    y[0] = (short int) ( temp );

    /* Shuffle values along one place for next time */

    y[2] = y[1]; /* y(n-2) = y(n-1) */
    y[1] = y[0]; /* y(n-1) = y(n) */
}
```

```

x[2] = x[1];    /* x(n-2) = x(n-1) */
x[1] = x[0];    /* x(n-1) = x(n)    */

/* temp is used as input next time through */
output_right_sample((short)(temp)); // output to DAC
return;

}

int main(void)
{
    L138_initialise_intr(FS_24000_HZ,ADC_GAIN_0DB,DAC_ATTEN_0DB,LCDK_LINE_INPUT);

    while(1);

} // end of main()

```

Result: Follow the steps used to observe the outputs for FIR filter.

Procedure to check the output

- After successful generation of .out file connect target to host PC
- Connect signal generator to LINE INPUT with 1Vp-p Sine Wave frequency with Frequency of 500Hz
- Connect CRO to LINE OUT with volt/div at 1 and time/div at 5msec
- Vary Frequency of Signal at Function Generator to See the Response (High Pass/Low Pass) according to Selected Filter Coefficients.

3. Write a C program to implement FIR filter for a given specification using TMS320C6748 (using overlap add approach)

C program to implement FIR filter(using overlap add approach)

```

#include "L138_LCDK_aic3106_init.h"

#include <math.h>
#include <stdio.h>

#define PTS 4096 //number of points used in FFT
#define NUMCOEFFS 31
#define PI 3.14159265358979

```

```

typedef struct { float real, imag;} COMPLEX;
void FFT(COMPLEX *Y, int N);

short buffercount = 0;           //number of new input samples in iobuffer
float in_buffer[PTS/2], out_buffer[PTS/2];           //primary input/output
buffer
COMPLEX samples[PTS];           //primary working buffer used by FFT
float overlap[PTS/2];           //intermediate result buffer
COMPLEX w[PTS];                 //twiddle factors stored in w
COMPLEX h[PTS];                 //freq domain filter coeffs stored in h
short i,flag;                   //general purpose index variable
float a,b;                      //variables used in complex multiply

float coeffs[NUMCOEFFS]={-0.01348909363, -0.01021953207,-
0.005696943495,2.920240591e-18, 0.006732751615, 0.01430734433, 0.02248182334,
0.03097631037, 0.03948523849, 0.04769114777, 0.05527933314, 0.06195262074,
0.06744547188, 0.0715367198, 0.0740602687, 0.07491308451, 0.0740602687,
0.0715367198, 0.06744547188, 0.06195262074, 0.05527933314, 0.04769114777,
0.03948523849, 0.03097631037, 0.02248182334, 0.01430734433,
0.006732751615,2.920240591e-18,-0.005696943495, -0.01021953207, -0.01348909363};
//LPF-Fc=2KHz Fs=4800KHz-Rectangular*/

interrupt void interrupt4(void) // interrupt service routine
{
    uint32 t sample;
    output_right_sample((short)(out_buffer[buffercount]));
    in_buffer[buffercount++] = (float)((short)input_right_sample());
    if (buffercount >= PTS/2)           //for overlap-add method iobuffer
    {                                   //is half size of FFT used
        buffercount = 0;
        flag = 1;
    }
    return;
}

int main(void)
{
    for (i=0 ; i<PTS ; i++)           //set up twiddle factors in array w
    {
        w[i].real = cos(PI*(i)/PTS);
        w[i].imag = -sin(PI*(i)/PTS);
    }
    printf("twiddle factors set up ...\n");
    printf("\n");                     //insert breakpoint
    for (i=0 ; i<PTS ; i++)           //set up complex time domain filter coeffs
    {
        h[i].real = 0.0;
        h[i].imag = 0.0;
    }
    for (i=0 ; i<NUMCOEFFS ; i++)
    {
        h[i].real = coeffs[i];
    }

    FFT(h,PTS);                       //transform filter coeffs to freq domain

    for (i=0 ; i<PTS/2 ; i++)

```

```

    {
        samples[i+PTS/2].real = 0.0;
        overlap[i] = 0.0;
    }

L138_initialise_intr(FS_48000_HZ, ADC_GAIN_0DB, DAC_ATTEN_0DB, LCDK_LINE_INPUT);

while(1)
{
    while (flag == 0);           //wait for iobuffer full
    flag = 0;
    for (i = 0 ; i<PTS/2 ; i++) //iobuffer into first half of
    {                             //samples buffer
        samples[i].real = in_buffer[i];
        out_buffer[i] = overlap[i]; //previously processed output
    }                             //to iobuffer
    for (i = 0 ; i<PTS/2 ; i++)
    {                             //copy second half of samples(previous
output) to overlap
        overlap[i] = samples[i+PTS/2].real;
        samples[i+PTS/2].real = 0.0; //zero-pad input from iobuffer
    }
    for (i=0 ; i<PTS ; i++)
        samples[i].imag = 0.0; //init imag parts in samples buffer
    FFT(samples,PTS);           //transform Input Samples to freq domain
    for (i=0 ; i<PTS ; i++)
    {
        a = samples[i].real;
        b = samples[i].imag;
        samples[i].real = h[i].real*a - h[i].imag*b;
        samples[i].imag = h[i].real*b + h[i].imag*a;
    }
    for (i=0 ; i<PTS ; i++) //inverse transform samples back to time
domain
    {
        samples[i].imag = -samples[i].imag;
    }
    FFT(samples,PTS);
    for (i=0 ; i<PTS ; i++)
    {
        samples[i].real = samples[i].real/PTS;
        samples[i].imag = -samples[i].imag/PTS;
    }

    for (i=0 ; i<PTS/2 ; i++) //Add first half of samples(present o/p)
with Second half of Previous output
        overlap[i] += samples[i].real; //to overlap
    }
}

void FFT(COMPLEX *Y, int N) //input sample array, # of points
{
    COMPLEX temp1,temp2; //temporary storage variables
    int i,j,k; //loop counter variables
    int upper_leg, lower_leg; //index of upper/lower butterfly leg
    int leg_diff; //difference between upper/lower leg

```

```

int num_stages=0;           //number of FFT stages, or iterations
int index, step;           //index and step between twiddle factor
i=1;                       //log(base 2) of # of points = # of stages
do
{
    num_stages+=1;
    i=i*2;
} while (i!=N);

leg_diff=N/2;              //starting difference between upper & lower legs
step=2;                    //step between values in twiddle.h
for (i=0;i<num_stages;i++) //for N-point FFT
{
    index=0;
    for (j=0;j<leg_diff;j++)
    {
        for (upper_leg=j;upper_leg<N;upper_leg+=(2*leg_diff))
        {
            lower_leg=upper_leg+leg_diff;
            temp1.real=(Y[upper_leg]).real + (Y[lower_leg]).real;
            temp1.imag=(Y[upper_leg]).imag + (Y[lower_leg]).imag;
            temp2.real=(Y[upper_leg]).real - (Y[lower_leg]).real;
            temp2.imag=(Y[upper_leg]).imag - (Y[lower_leg]).imag;
            (Y[lower_leg]).real=temp2.real*(w[index]).real-temp2.imag*(w[index]).imag;
            (Y[lower_leg]).imag=temp2.real*(w[index]).imag+temp2.imag*(w[index]).real;
            (Y[upper_leg]).real=temp1.real;
            (Y[upper_leg]).imag=temp1.imag;
        }
        index+=step;
    }
    leg_diff=leg_diff/2;
    step*=2;
}
j=0;
for (i=1;i<(N-1);i++)      //bit reversal for resequencing data*/
{
    k=N/2;
    while (k<=j)
    {
        j=j-k;
        k=k/2;
    }
    j=j+k;
    if (i<j)
    {
        temp1.real=(Y[j]).real;
        temp1.imag=(Y[j]).imag;
        (Y[j]).real=(Y[i]).real;
        (Y[j]).imag=(Y[i]).imag;
        (Y[i]).real=temp1.real;
        (Y[i]).imag=temp1.imag;
    }
}
return;                    //end of FFT()
}

```

Result: Follow the steps used to observe the outputs for FIR filter.

Procedure to check the output

- After successful generation of .out file connect target to host PC
- Connect signal generator to LINE INPUT with 1Vp-p Sine Wave frequency with Frequency of 500Hz
- Connect CRO to LINE OUT with volt/div at 1 and time/div at 5msec
- Vary Frequency of Signal at Function Generator to See the Response (High Pass/Low Pass) according to Selected Filter Coefficients.

Exercise:

1. Design and implement a bandpass and bandreject IIR filters for a given specification using a sampling frequency of 48Khz.

L11-12: Mini Project

Mini project should be relevant to the laboratory. This means it should have aspect of either control system or digital signal processing.